

Hilbert Transform Based Demodulator Implementation

Introduction

The Hilbert transform based demodulator is a powerful signal processing technique used for envelope detection in amplitude modulated (AM) signals and various other applications. This document provides a comprehensive guide to implementing this method, which relies on creating an analytic signal representation of the input signal.

Envelope detection is the process of extracting the amplitude variations (or envelope) of a modulated signal. This technique is essential in communications systems, particularly for AM demodulation, biomedical signal processing, and various instrumentation applications where the signal envelope contains the information of interest.

This document presents a comprehensive implementation of a Hilbert transform based AM demodulator, progressing from theoretical foundations through simulation to hardware deployment on a Zynq UltraScale+ FPGA platform. The design begins in Simulink with envelope detection using Hilbert transformers and is systematically converted to Vitis HLS C++ code, implementing each component—Hilbert transformer, delay line, envelope detector, lowpass filter, downsampler, and DC removal—as individual, verified modules. The complete demodulator processes a 100 kHz carrier with 1 kHz message signal at 480 kHz sampling rate, achieving decimation to 32 kHz output. Verification proceeds through multiple levels: HLS testbenches with Python visualization, Vivado simulations using AXI Stream VIPs, system-level validation with Zynq UltraScale+ VIP including DMA transfers, and finally deployment on an Ultra96 V2 board running PetaLinux. The implementation demonstrates successful integration with Linux userspace DMA control, processing Simulink-generated test signals and producing demodulated audio output, confirming end-to-end functionality from algorithm to embedded system deployment.

Theoretical Foundation

The Analytic Signal

An analytic signal is a complex-valued representation of a real signal that contains no negative frequency components. For a real signal $x(t)$, the analytic signal $z(t)$ is defined as:

$$z(t) = x(t) + j \cdot H\{x(t)\}$$

where:

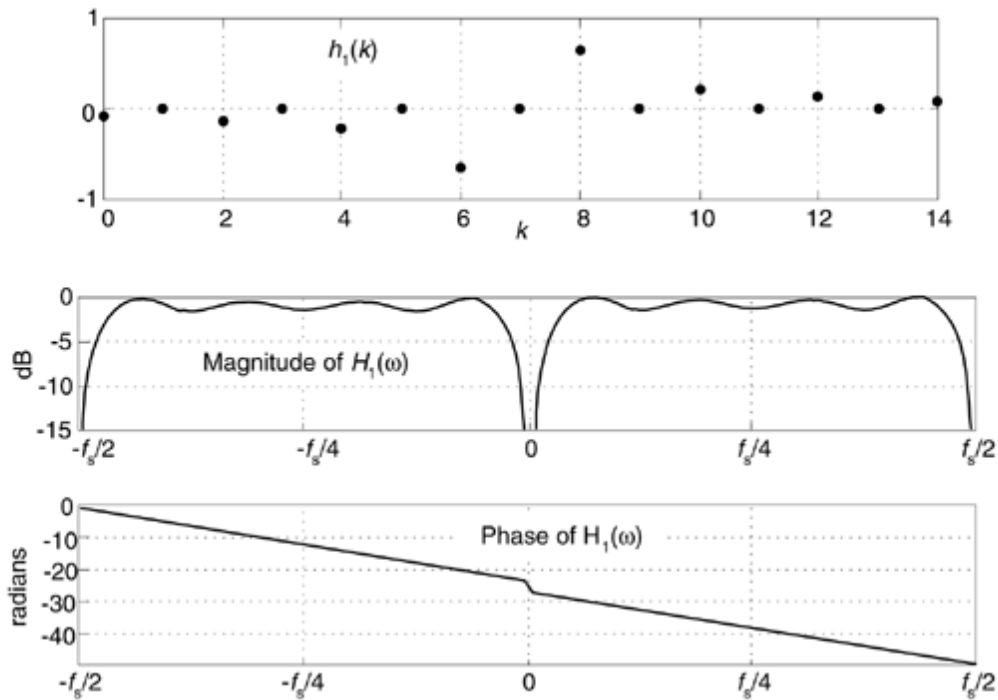
- $x(t)$ is the original real signal
- $H\{x(t)\}$ is the Hilbert transform of $x(t)$
- j is the imaginary unit ($\sqrt{-1}$)

The Hilbert Transform

The Hilbert transform is a linear operator that produces a 90-degree phase shift of all frequency components of a signal. In the frequency domain, it can be defined as:

$$H\{x(t)\} = x(t) * h(t)$$

where $h(t) = 1/(\pi t)$ is the impulse response of the Hilbert transformer, and $*$ denotes convolution.



Envelope Extraction

Once the analytic signal is formed, the envelope $e(t)$ of the original signal $x(t)$ is simply the magnitude of the analytic signal:

$$e(t) = |z(t)| = \sqrt{[x(t)]^2 + H\{x(t)\}^2}$$

This mathematical relationship provides the foundation for envelope detection using the Hilbert transform method.

Implementation Approach

Step 1: Design the Hilbert Transformer

The Hilbert transformer is typically implemented as a finite impulse response (FIR) filter. A common approach uses the Parks-McClellan algorithm to design an optimal FIR filter approximation of the ideal Hilbert transformer.

Step 2: Compensate for Filter Delay

Since the FIR Hilbert transformer introduces a delay of $N/2$ samples, the original signal must be delayed by the same amount to maintain proper phase alignment when forming the analytic signal.

Step 3: Form the Analytic Signal

Combine the delayed original signal with the Hilbert-transformed signal to create the complex analytic signal:

$$z[n] = x[n - N/2] + j \cdot H\{x[n]\}$$

where:

- $z[n]$ is the analytic signal
- $x[n - N/2]$ is the delayed original signal
- $H\{x[n]\}$ is the Hilbert transform output

Step 4: Calculate the Envelope

Compute the magnitude (absolute value) of the complex analytic signal:

$$e[n] = |z[n]| = \sqrt{[\text{Re}\{z[n]\}]^2 + [\text{Im}\{z[n]\}]^2}$$

This operation extracts the instantaneous amplitude envelope of the signal.

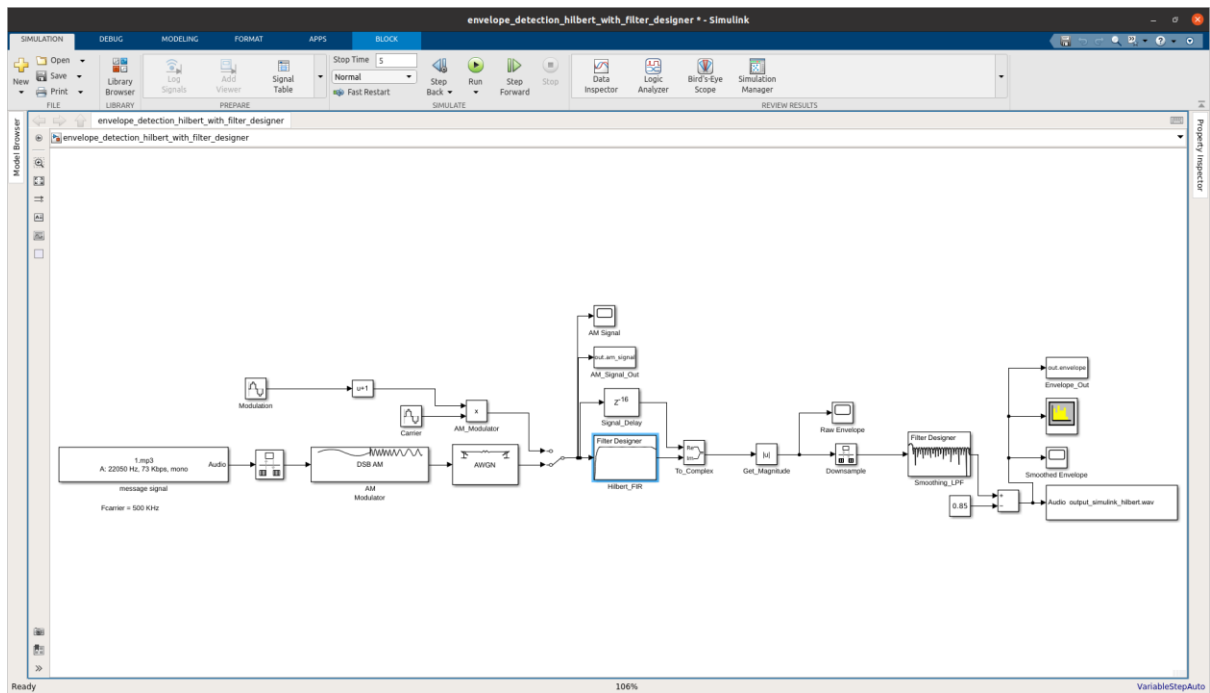
Step 5: Post-Processing (Optional)

After envelope extraction, additional processing may improve results:

- **Downsampling**
- **Lowpass Filtering**

THE STARTING SIMULINK MODEL

A simulink model has been prepared to practically test the Hilbert transform based AM demodulation:

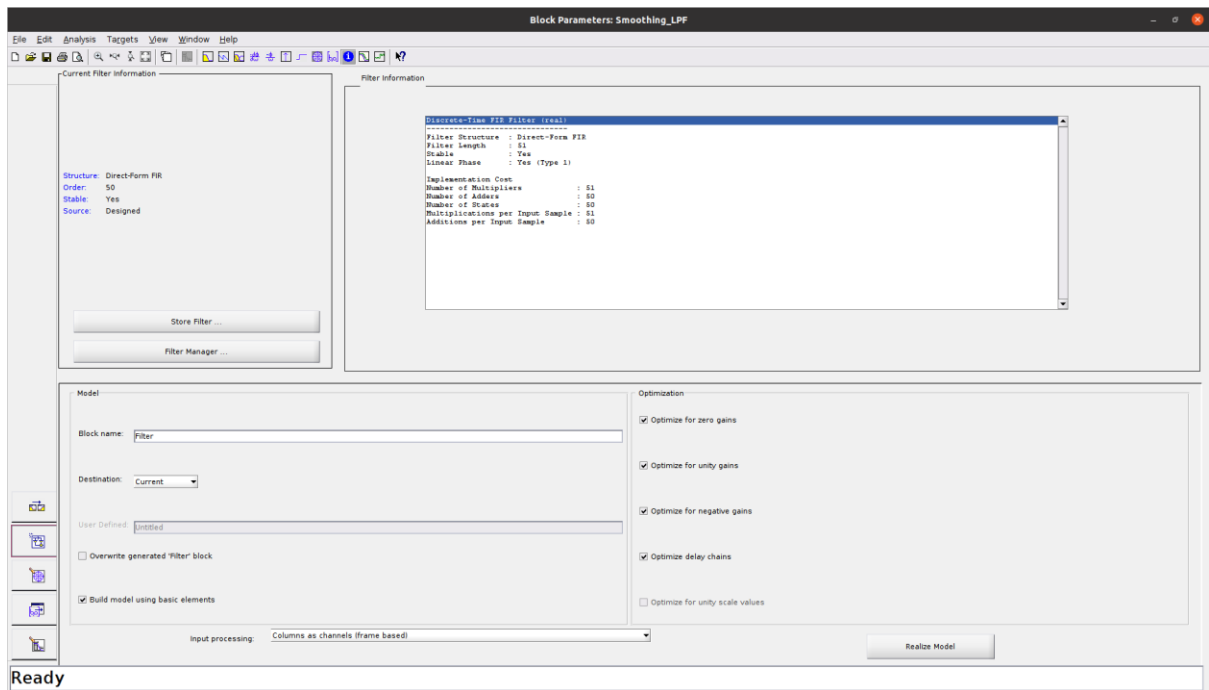
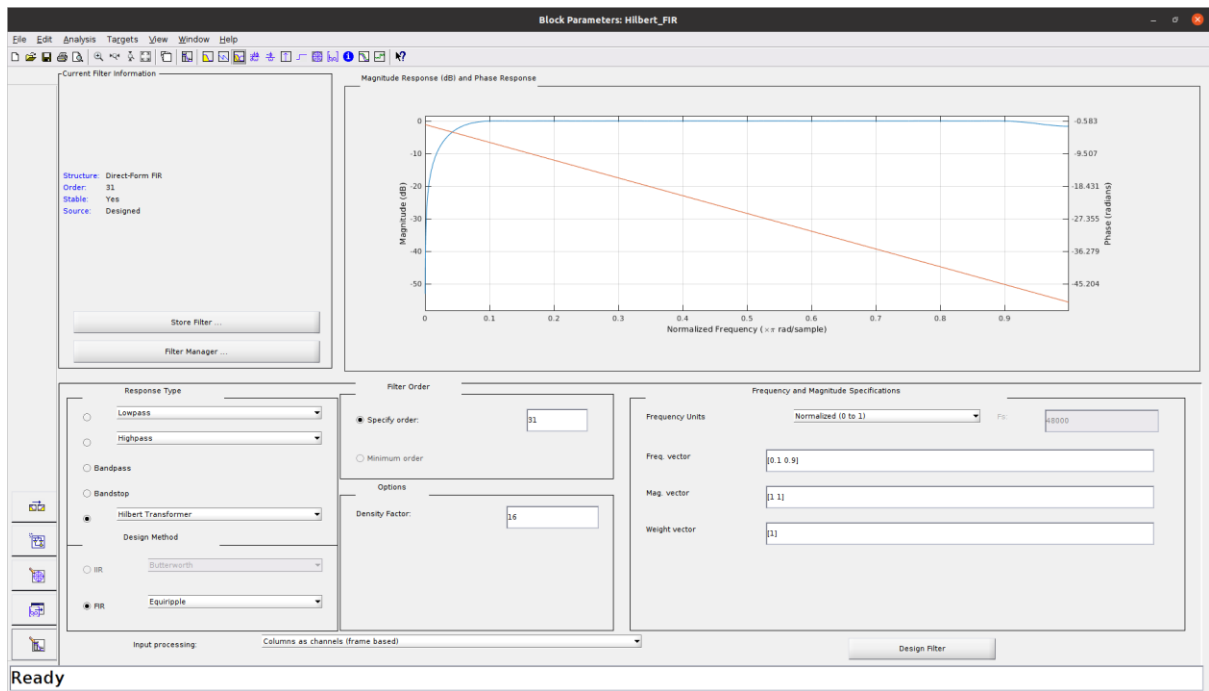


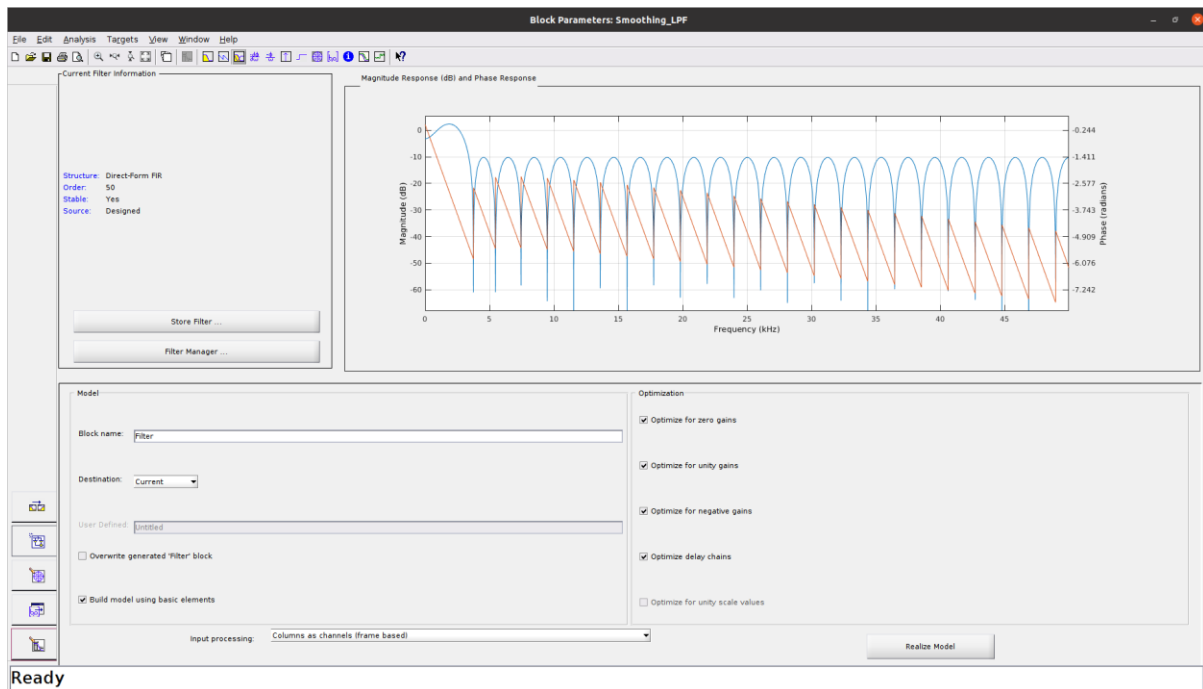
The two filters in use, have the following configuration settings:

The 'Block Parameters: Hilbert_FIR' dialog box displays the configuration for the Hilbert_FIR filter. The 'Current Filter Information' section shows the filter is a 'Direct-Form FIR' with an order of 31, which is stable and designed. The 'Filter Information' section provides detailed specifications:

- Filter Structure: Direct-Form FIR
- Filter Length: 32
- Stable: Yes
- Linear Phase: Yes (Type 4)
- Implementation Cost: 32
- Number of Multipliers: 31
- Number of Adders: 31
- Number of States: 31
- Multiplications per Input Sample: 32
- Additions per Input Sample: 31

The 'Response Type' section shows the filter is configured as a 'Lowpass' filter. The 'Filter Order' section shows the order is specified as 31. The 'Options' section shows the 'Density Factor' is set to 16. The 'Frequency and Magnitude Specifications' section shows the frequency units are 'Normalized (0 to 1)' with a passband from 0 to 0.9 and a stopband starting at 1. The 'Design Filter' button is located at the bottom right.



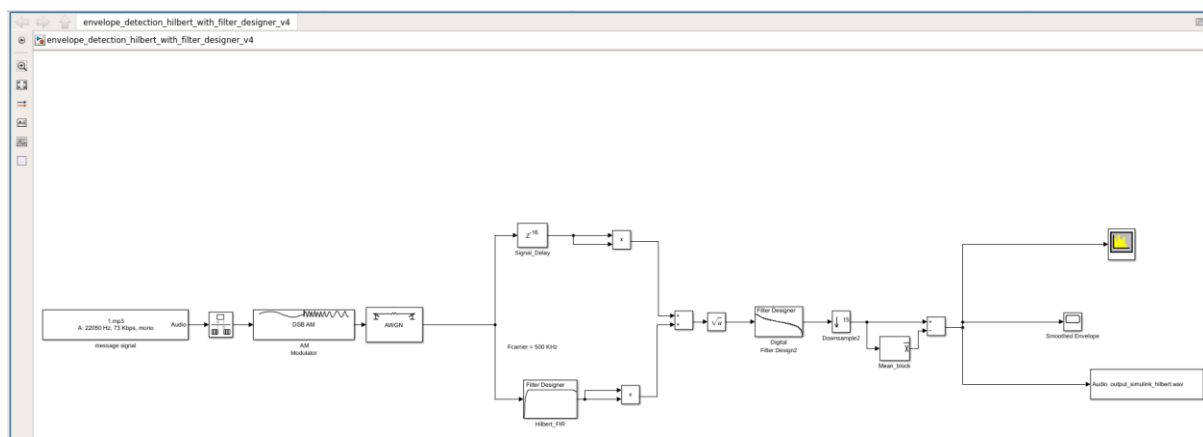


The model follows the one being shown on the Mathworks post:

<https://uk.mathworks.com/help/dsp/ug/envelope-detection.html>

THE FINAL SIMULINK MODEL

The starting Simulink model has been reworked a bit, so to be more easily implemented in Vitis HLS:



The complex number magnitude has been computed by using explicitly adders, multipliers and square root blocks.

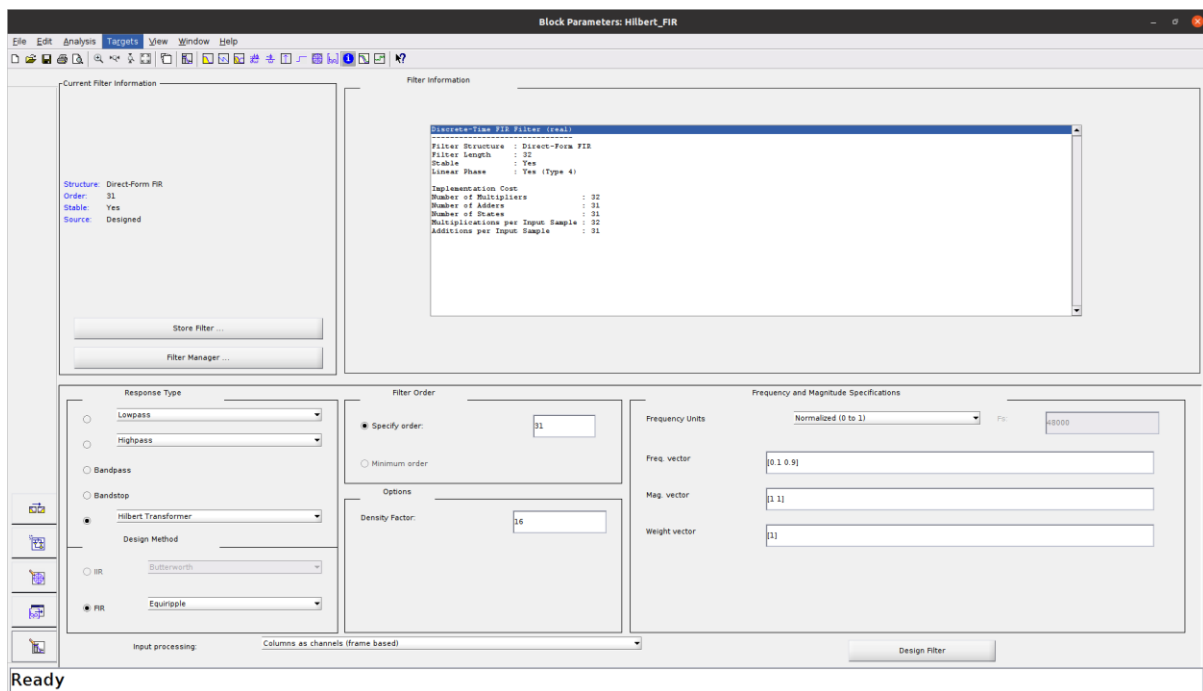
The downsampler block has been replaced with a simpler one.

A mean block has been introduced to remove the DC bias in the demodulated signal.

This, with the aim to design one by one these simpler blocks in Vitis HLS.

THE SIMULINK MODEL: GENERATING THE VHDL FILTERS

Starting from the Simulink model, opening the block configuration panel and pressing on targets -> generate HDL:



We get:

Generate HDL (Direct-Form FIR, order = 31)

Set Basic Options

LanguageVHDL

Name: hilbert_transform

FolderhdlsrcBrowse...☐ Generate MATLAB code

Filter Architecture

Global Settings

Test Bench

EDA Tool Scripts

Architecture:

Fully parallel

Folding factor: 1

Multiplier: 32

Coefficient source:

Internal

Coefficient multipliers:

Multiplier

Multiplier input pipeline:

0

Multiplier output pipeline:

0

☐ Add pipeline registers

FIR adder style:

Linear

☐ Optimize for HDL

?

Generate

Close

Help

Generate HDL (Direct-Form FIR, order = 31)

Set Basic Options

Language

VHDL

Name:

hilbert_transform

Folder

hdlsrc

Browse...

☐ Generate MATLAB code

Filter Architecture

Global Settings

Test Bench

EDA Tool Scripts

Reset type

Asynchronous

Reset asserted level

Active-high

Clock input port

clk

Clock enable input port

clk_enable

Reset input port

reset

Clock inputs:

Single

Remove reset from:

None

Additional settings

General

Ports

Advanced

Comment in header

Verilog file extension

.v

VHDL file extension

.vhd

Entity conflict postfix

_block

Package postfix

_pkg

Reserved word postfix

_rsvd

Split entity file postfix

_entity

Clocked process postfix

_process

Split arch file postfix

_arch

Complex real part postfix

_re

☐ Split entity and architecture

Complex imaginary part postfix

_im

Vector prefix:

vector_of_

Coefficient prefix:

coeff

Instance prefix:

u_

?

Generate

Close

Help

Generate HDL (Direct-Form FIR, order = 31)

Set Basic Options

Language

VHDL

Name:

hilbert_transform

Folder

hdlsrc

Browse...

☐ Generate MATLAB code

Filter Architecture

Global Settings

Test Bench

EDA Tool Scripts

Test Bench Generation Output

☒ HDL test bench

Test bench language:

VHDL

File name:

filter_tb

☐ Cosimulation blocks

☐ Cosimulation model

Mentor Graphics ModelSim

Stimuli

Configuration

☒ Impulse response

☒ Step response

☒ Ramp response

☒ Chirp response

☒ White noise response

☐ User defined response

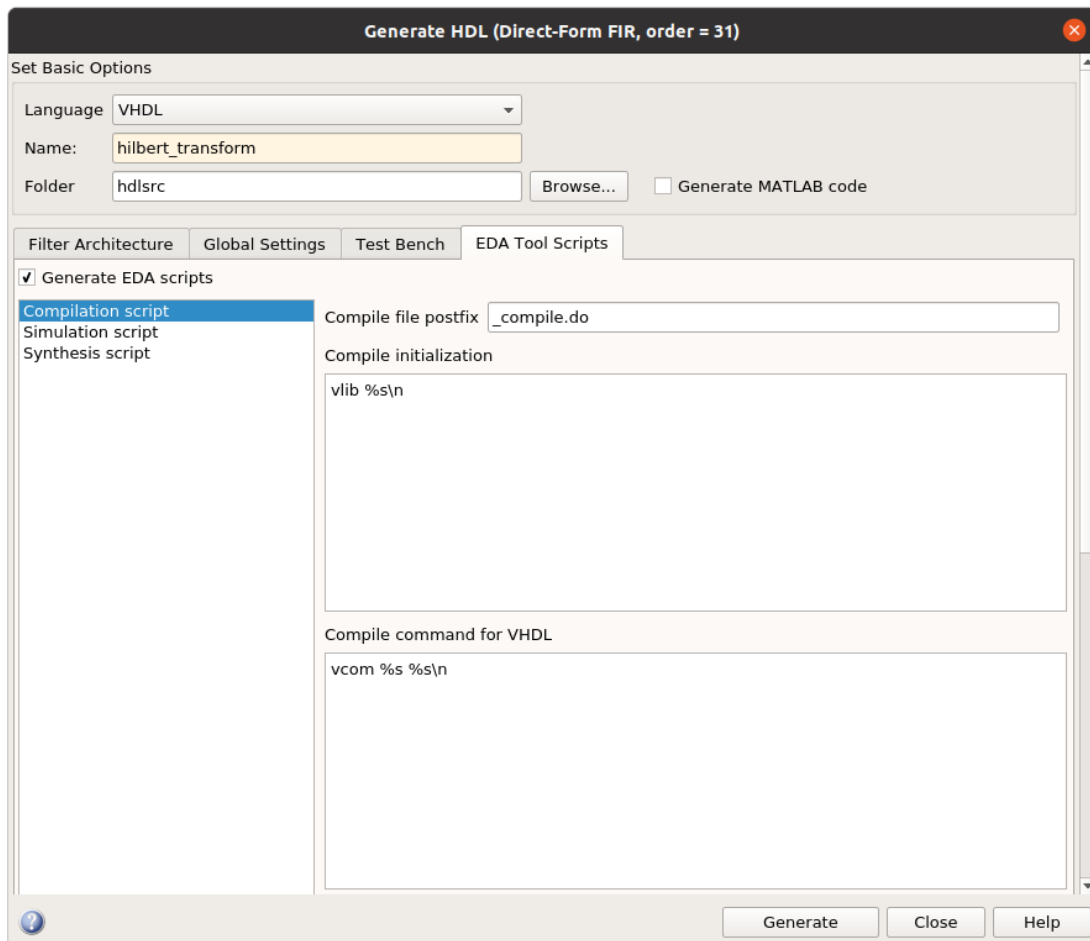
User stimulus:

?

Generate

Close

Help



It gives us VHDL code where floating point arithmetic is used:

```

1  hilbert_transform.vhd  ProgrammableFIRFilterForFPGAExample.m  +
8  -----
9  -- HDL Code Generation Options:
10 --
11 -- TargetLanguage: VHDL
12 -- Name: hilbert_transform
13 -- TestBenchStimulus: impulse step ramp chirp noise
14
15 -----
16 -- HDL Implementation      : Fully parallel
17 -- Folding Factor          : 1
18 -----
19 -- Filter Settings:
20 --
21 -- Discrete-Time FIR Filter (real)
22 -----
23 -- Filter Structure      : Direct-Form FIR
24 -- Filter Length        : 32
25 -- Stable                 : Yes
26 -- Linear Phase          : Yes (Type 4)
27 -----
28
29
30
31 LIBRARY IEEE;
32 USE IEEE.std_logic_1164.all;
33 USE IEEE.numeric_std.ALL;
34
35 ENTITY hilbert_transform IS
36     PORT( clk           : IN    std_logic;
37           clk_enable    : IN    std_logic;
38           reset         : IN    std_logic;
39           filter_in      : IN    real; -- double
40           filter_out     : OUT   real  -- double
41     );
42
43 END hilbert_transform;
44
45
46 -----
47 --Module Architecture: hilbert_transform
48 -----
49 ARCHITECTURE rtl OF hilbert_transform IS
50     -- Local Functions
51     -- Type Definitions
52     TYPE delay_pipeline_type IS ARRAY (NATURAL range <>) OF real; -- double
53     -- Constants
54     CONSTANT coeff1      : real := -3.0838958070072532E-03; -- double
55     CONSTANT coeff2      : real := -1.1563322168801279E-03; -- double
56     CONSTANT coeff3      : real := -5.9708316939980901E-03; -- double
57     CONSTANT coeff4      : real := -3.3653501813806982E-03; -- double
58     CONSTANT coeff5      : real := -1.1370294595527887E-02; -- double
59     CONSTANT coeff6      : real := -7.5683440589330679E-03; -- double
60     CONSTANT coeff7      : real := -1.9628515643984545E-02; -- double
61     CONSTANT coeff8      : real := -1.4978186105303284E-02; -- double
62     CONSTANT coeff9      : real := -3.2113551266464938E-02; -- double
63     CONSTANT coeff10     : real := -2.7894786623769277E-02; -- double

```

```
151 SIGNAL sum30 : real := 0.0; -- double
152 SIGNAL sum31 : real := 0.0; -- double
153 SIGNAL output_register : real := 0.0; -- double
154
155
156 BEGIN
157
158 -- Block Statements
159 Delay_Pipeline_process : PROCESS (clk, reset)
160 BEGIN
161 IF reset = '1' THEN
162 delay_pipeline(0 TO 31) <= (OTHERS => 0.0000000000000000E+00);
163 ELSIF clk'event AND clk = '1' THEN
164 IF clk_enable = '1' THEN
165 delay_pipeline(0) <= filter_in;
166 delay_pipeline(1 TO 31) <= delay_pipeline(0 TO 30);
167 END IF;
168 END IF;
169 END PROCESS Delay_Pipeline_process;
170
171 product32 <= delay_pipeline(31) * coeff32;
172
173 product31 <= delay_pipeline(30) * coeff31;
174
175 product30 <= delay_pipeline(29) * coeff30;
176
177 product29 <= delay_pipeline(28) * coeff29;
178
179 product28 <= delay_pipeline(27) * coeff28;
180
181 product27 <= delay_pipeline(26) * coeff27;
182
183 product26 <= delay_pipeline(25) * coeff26;
184
185 product25 <= delay_pipeline(24) * coeff25;
186
187 product24 <= delay_pipeline(23) * coeff24;
188
189 product23 <= delay_pipeline(22) * coeff23;
190
191 product22 <= delay_pipeline(21) * coeff22;
192
193 product21 <= delay_pipeline(20) * coeff21;
194
195 product20 <= delay_pipeline(19) * coeff20;
196
197 product19 <= delay_pipeline(18) * coeff19;
198
199 product18 <= delay_pipeline(17) * coeff18;
200
201 product17 <= delay_pipeline(16) * coeff17;
202
203 product16 <= delay_pipeline(15) * coeff16;
204
205 product15 <= delay_pipeline(14) * coeff15;
```

THE HLS HILBERT TRANSFORMER

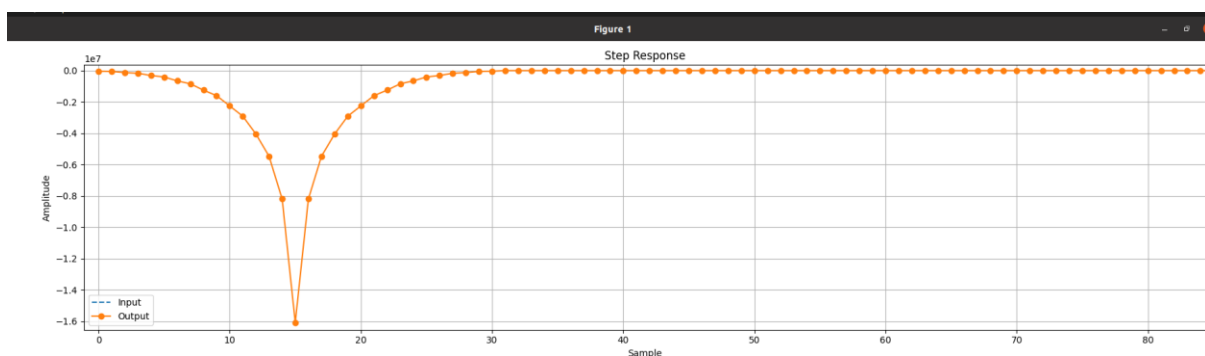
The Hilbert transformer coefficients generated in Simulink, have been turned in the following unoptimized C++ HLS code:

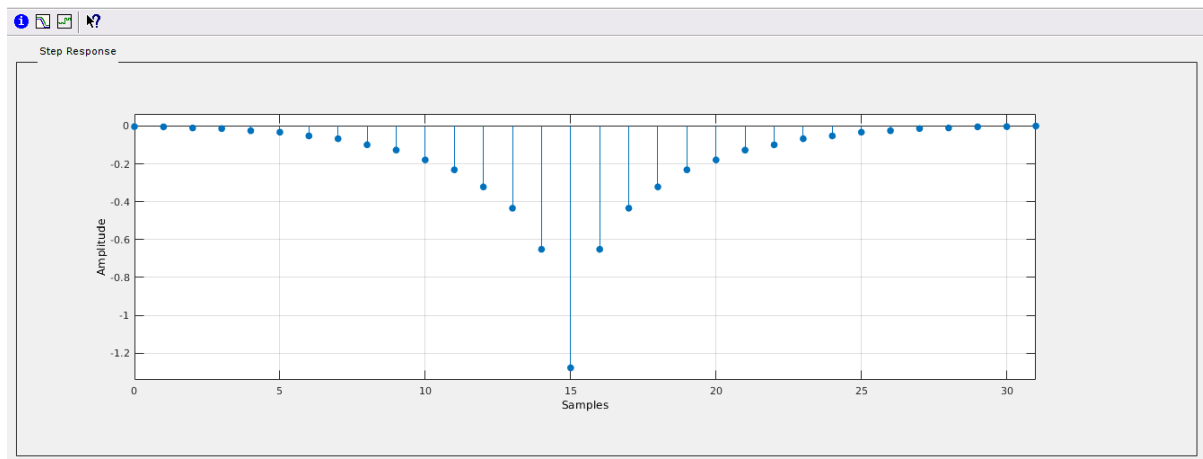
```
Synthesis Summary(solution1)  filter1.cpp  filter1.hpp x
1 #include <ap_int.h>
2 #include <ap_fixed.h>
3 #include <math.h>
4
5 #define FILTER_LENGTH 32
6
7 // Data format
8 const int DataWordSize = 26;
9 const int DataIntSize = 4;
10 const float DataMaxVal = pow(2.0, DataIntSize-1);
11 typedef ap_fixed<DataWordSize, DataIntSize, AP_RND, AP_SAT, 0> data_type;
12
13
14
15 //coefficient constants and data types
16 const int CoeffWordSize = 18;
17 const int CoeffIntSize = 4;
18 typedef ap_fixed<CoeffWordSize, CoeffIntSize, AP_RND, AP_SAT, 0> coeff_type;
19
20
21 // Filter coefficients
22 static const coeff_type coeffs[FILTER_LENGTH] = {
23     -3.0838958070072532E-03, // coeff1
24     -1.1563322168801279E-03, // coeff2
25     -5.9708316939980901E-03, // coeff3
26     -3.3653501813806982E-03, // coeff4
27     -1.1370294595527887E-02, // coeff5
28     -7.5683440589330679E-03, // coeff6
29     -1.9628515643984545E-02, // coeff7
30     -1.4978186105303284E-02, // coeff8
31     -3.2113551266464938E-02, // coeff9
32     -2.7894786623769277E-02, // coeff10
33     -5.2056692551864792E-02, // coeff11
34     -5.2256610273355014E-02, // coeff12
35     -9.0302534572430176E-02, // coeff13
36     -1.1258848799869187E-01, // coeff14
37     -2.1700314947952859E-01, // coeff15
38     -6.2655534028311499E-01, // coeff16
39     6.2655534028311499E-01, // coeff17
40     2.1700314947952859E-01, // coeff18
41     1.1258848799869187E-01, // coeff19
42     9.0302534572430176E-02, // coeff20
43     5.2256610273355014E-02, // coeff21
44     5.2056692551864792E-02, // coeff22
45     2.7894786623769277E-02, // coeff23
46     3.2113551266464938E-02, // coeff24
47     1.4978186105303284E-02, // coeff25
```

```
48     1.9628515643984545E-02, // coeff26
49     7.5683440589330679E-03, // coeff27
50     1.1370294595527887E-02, // coeff28
51     3.3653501813806982E-03, // coeff29
52     5.9708316939980901E-03, // coeff30
53     1.1563322168801279E-03, // coeff31
54     3.0838958070072532E-03, // coeff32
55 };
56
57
58 //Accumulator data types
59 typedef ap_fixed<DataWordSize+CoeffWordSize+6, DataIntSize+CoeffIntSize, AP_TRN, AP_WRAP, 0> accum_type;
60
61 // Processes one sample through the fixed-point filter
62 data_type filter1(data_type input);
63
64
65
66
```

```
Synthesis Summary(solution1)  filter1.cpp x  filter1.hpp
1  #include "filter1.hpp"
2
3
4
5  // Section 2: static internal state
6  data_type filter1(data_type input) {
7      static bool initialized = false;
8      static data_type delay_pipeline[FILTER_LENGTH];
9      static data_type d2;
10     static accum_type acc = 0.0;
11
12
13     // One-time initialization
14     if (!initialized) {
15         acc = 0.0;
16         for(int i = 0; i < FILTER_LENGTH; i ++ )
17             delay_pipeline[i] = 0.0;
18         initialized = true;
19     }
20
21
22     // Shift the delay line
23     for (int i = FILTER_LENGTH - 1; i > 0; i--)
24         delay_pipeline[i] = delay_pipeline[i - 1];
25     delay_pipeline[0] = input;
26
27     // Compute convolution (dot product)
28     acc = 0.0; //reset accumulator
29     for (int i = 0; i < FILTER_LENGTH; i++)
30         acc += delay_pipeline[i] * coeffs[i];
31
32     return acc;
33 }
34
35
```

By running the testbench, we can see that the impulse response plotted in Python matches the one found in Simulink:





So the code to be deployed on the FPGA, is a good match of the starting Simulink counterpart.

THE HLS DELAY LINE

The source code of the delay line follows:

```
Synthesis Summary(solution1)  delay_line.cpp x
1
2 #include "delay_line.hpp"
3
4 data_type delay_line(data_type filter_in) {
5     const int DELAY_SAMPLES = 16;
6     static data_type buffer[DELAY_SAMPLES] = {0};
7     static int index = 0;
8
9     // Get the delayed output (oldest sample in buffer)
10    data_type output = buffer[index];
11
12    // Store new input sample
13    buffer[index] = filter_in;
14
15    // Update circular buffer index
16    index = (index + 1) % DELAY_SAMPLES;
17
18    return output;
19 }
```

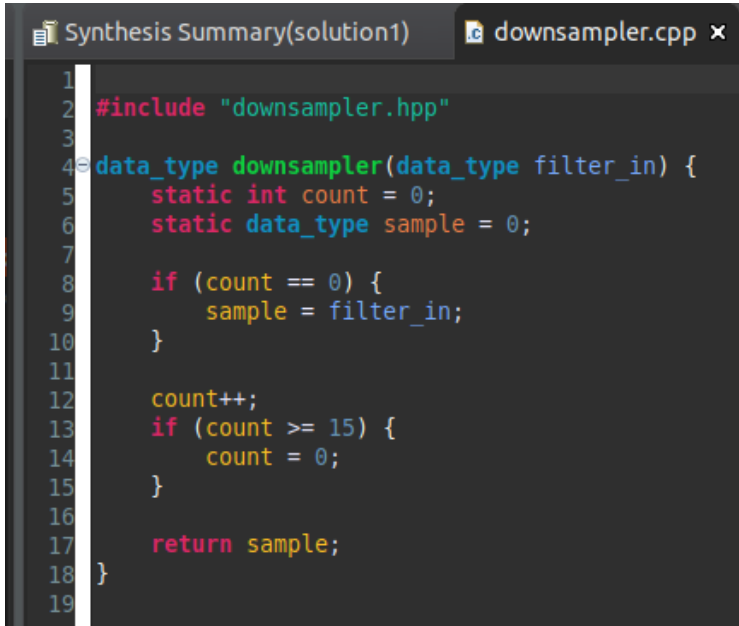
A testbench has been created: it verifies the correct operation of a 16-sample delay line implemented in `delay_line()`.

It runs two tests: an impulse response, where a single pulse at sample 0 confirms the 16-sample delay when it reappears at the output, and a sine wave test, where a 32-sample-period sine input demonstrates the expected 180° phase shift caused by the 16-sample delay.

The printed tables show input, output, and expected or difference values, confirming that the delay line introduces the intended fixed delay.

THE HLS DOWNSAMPLER

The downsampler has been coded as shown in the following:



```
1
2 #include "downsampler.hpp"
3
4 data_type downsampler(data_type filter_in) {
5     static int count = 0;
6     static data_type sample = 0;
7
8     if (count == 0) {
9         sample = filter_in;
10    }
11
12    count++;
13    if (count >= 15) {
14        count = 0;
15    }
16
17    return sample;
18 }
19
```

A testbench that verifies the operation of the downsampler with a decimation factor of 15 has been written.

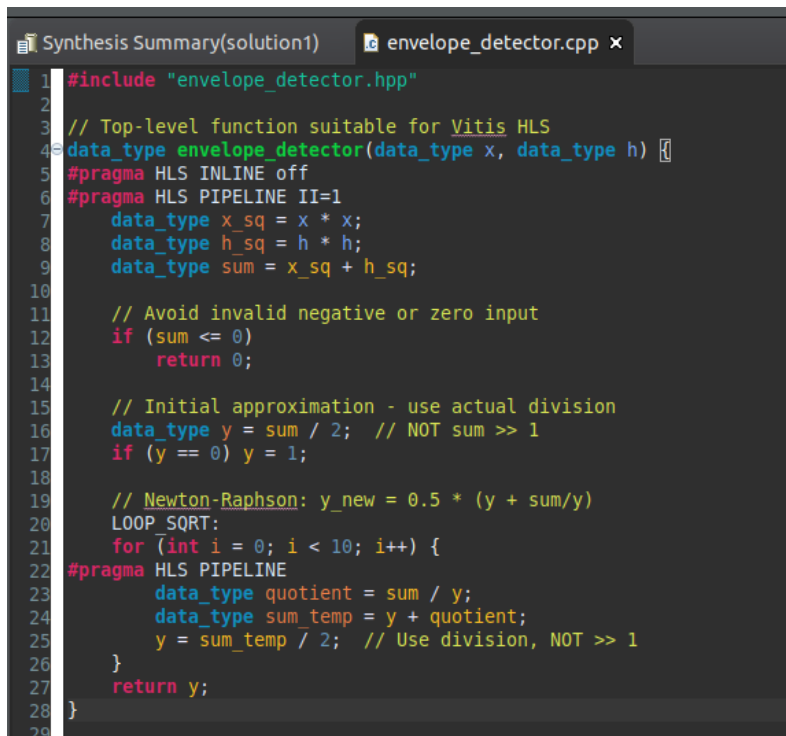
It performs two tests: a ramp input, confirming that the output updates only once every 15 samples while holding constant in between, and a sine wave input, demonstrating that the signal is correctly sampled at one-fifteenth of the input rate.

The printed tables show the input, output, and whether an update occurred, confirming that the effective output rate is the input sample rate divided by 15.

THE HLS ENVELOPE DETECTOR

The code below computes the magnitude of a complex (analytic) signal given its real part x and imaginary/quadrature part h , returning $\sqrt{x^2 + h^2}$.

The square root is approximated using an iterative Newton-Raphson method for efficient hardware implementation.



```
1 #include "envelope_detector.hpp"
2
3 // Top-level function suitable for Vitis HLS
4 data_type envelope_detector(data_type x, data_type h) {
5     #pragma HLS INLINE off
6     #pragma HLS PIPELINE II=1
7     data_type x_sq = x * x;
8     data_type h_sq = h * h;
9     data_type sum = x_sq + h_sq;
10
11     // Avoid invalid negative or zero input
12     if (sum <= 0)
13         return 0;
14
15     // Initial approximation - use actual division
16     data_type y = sum / 2; // NOT sum >> 1
17     if (y == 0) y = 1;
18
19     // Newton-Raphson: y_new = 0.5 * (y + sum/y)
20     LOOP_SQRT:
21     for (int i = 0; i < 10; i++) {
22     #pragma HLS PIPELINE
23         data_type quotient = sum / y;
24         data_type sum_temp = y + quotient;
25         y = sum_temp / 2; // Use division, NOT >> 1
26     }
27     return y;
28 }
```

The testbench used, verifies the envelope_detector function using static, hardcoded inputs.

The real part and the imaginary/quadrature part are fixed values, and the expected output is also hardcoded.

The test calls the envelope detector and prints the inputs, computed result, and expected value to confirm correct computation of the signal envelope.

THE HLS LOWPASS FILTER

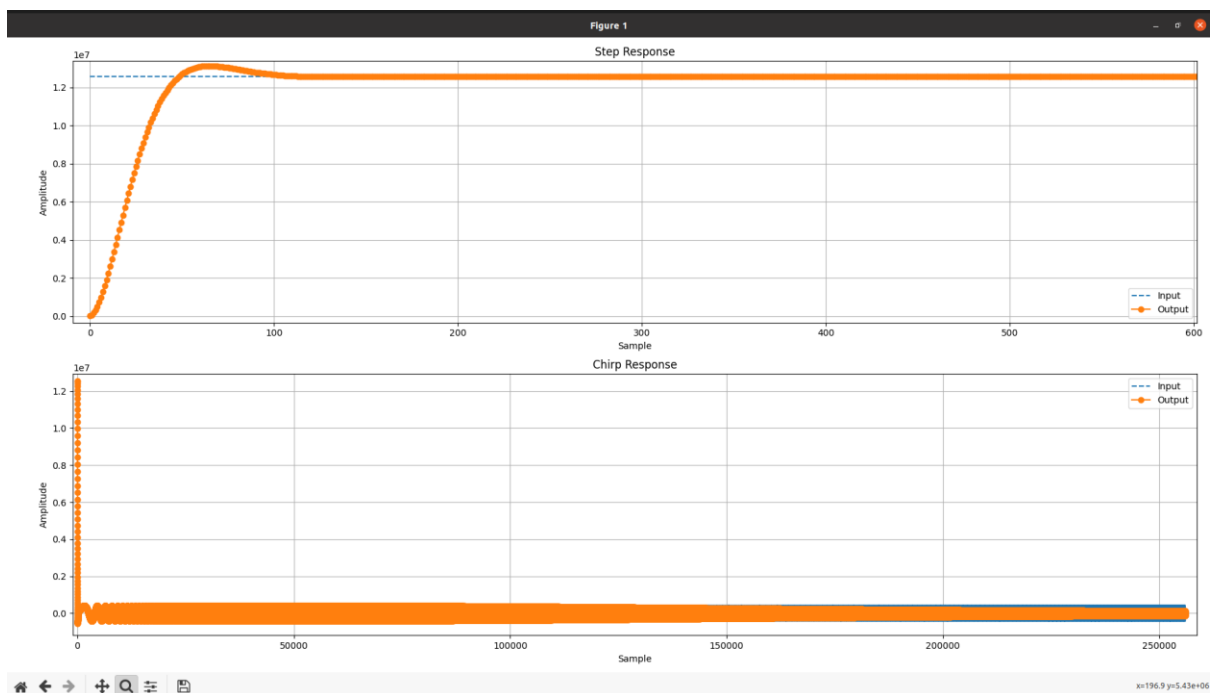
The LPF code is:

```

1
2 #include "filter_envelope_detector.hpp"
3
4 data_type applyIIRFilter(data_type filter_in) {
5     static bool initialized = false;
6     static data_type delay0;
7     static data_type delay1;
8     static data_type output_register;
9
10    // One-time initialization
11    if (!initialized) {
12        delay0 = 0.0;
13        delay1 = 0.0;
14        output_register = 0.0;
15        initialized = true;
16    }
17
18    // Signal path logic
19    data_type input_register = filter_in;
20    accum_type scale1 = input_register * scaleconst1;
21    accum_type inputconv1 = scale1;
22
23    accum_type a2mul1 = delay0 * coeff_a2_section1;
24    accum_type a3mul1 = delay1 * coeff_a3_section1;
25    accum_type b2mul1 = delay0 * coeff_b2_section1;
26
27    accum_type a2sum1 = inputconv1 - a2mul1;
28    accum_type a1sum1 = a2sum1 - a3mul1;
29    accum_type b2sum1 = a1sum1 + b2mul1;
30    accum_type b1sum1 = b2sum1 + delay1;
31
32    output_register = b1sum1;
33
34    // Update delay line
35    delay1 = delay0;
36    delay0 = a1sum1;
37
38    //return output sample
39    return output_register;
40 }
41
42

```

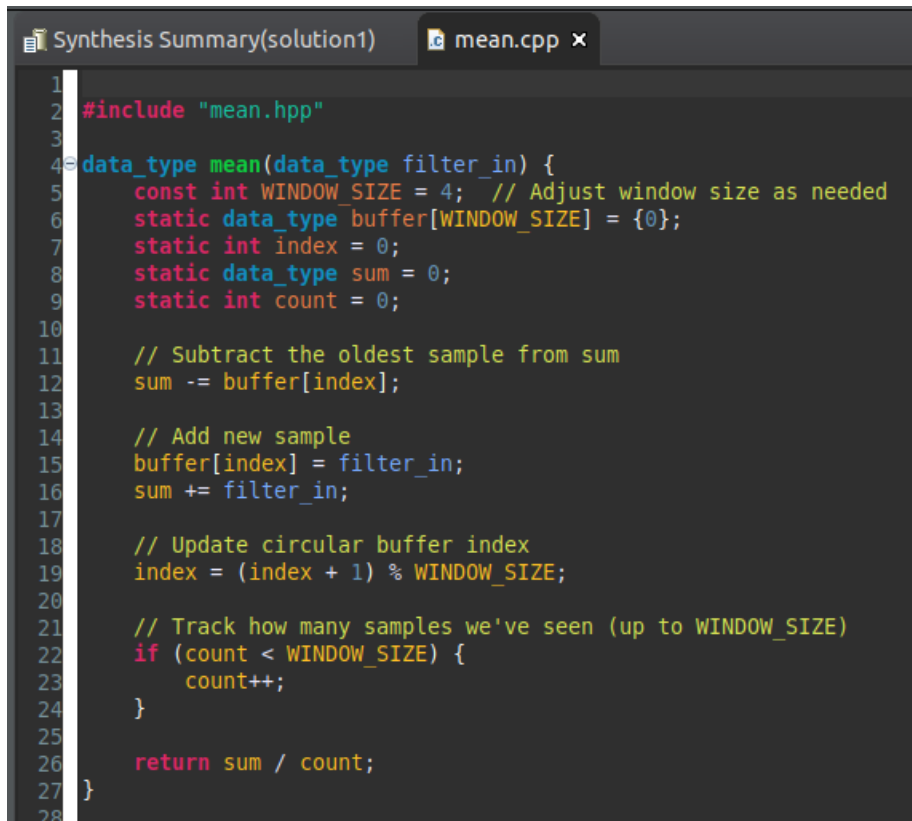
The testbench computes the step and chirp response:



And they match the Matlab ones.

THE HLS MEAN

The mean value is computed as:



```
1
2 #include "mean.hpp"
3
4 data_type mean(data_type filter_in) {
5     const int WINDOW_SIZE = 4; // Adjust window size as needed
6     static data_type buffer[WINDOW_SIZE] = {0};
7     static int index = 0;
8     static data_type sum = 0;
9     static int count = 0;
10
11     // Subtract the oldest sample from sum
12     sum -= buffer[index];
13
14     // Add new sample
15     buffer[index] = filter_in;
16     sum += filter_in;
17
18     // Update circular buffer index
19     index = (index + 1) % WINDOW_SIZE;
20
21     // Track how many samples we've seen (up to WINDOW_SIZE)
22     if (count < WINDOW_SIZE) {
23         count++;
24     }
25
26     return sum / count;
27 }
28
```

The testbench used runs three tests: a constant input to confirm that the mean equals the constant value, a ramp input to check correct averaging as the window fills and slides, and an alternating input to observe convergence toward the midpoint value.

The inputs and expected results are printed to validate correct moving-average behaviour.

THE HLS AM DEMODULATOR

The AM demodulator leverages upon all the IP cores we have looked so far:

```
Synthesis Summary(solution1)  am_demodulator.cpp x
144 data_type am_demodulator(data_type filter_in) {
145 #pragma HLS INLINE off
146 #pragma HLS DATAFLOW
147
148
149 data_type delayed_signal = 0.0;
150 data_type hilbert_signal = 0.0;
151 data_type envelope = 0.0;
152 data_type filtered_envelope = 0.0;
153 data_type downsampled_output = 0.0;
154 data_type mean_val = 0.0;
155 data_type final_output = 0.0;
156 const data_type gain1 = 1.0E+00;
157 const data_type gain2 = 1.0E+00;
158 const data_type gain3 = 1.0E+00;
159
160
161 filter_in = filter_in * gain1 * gain2;
162
163 // Step 1: Delay input by 16 samples
164 delayed_signal = delay_line(filter_in);
165
166 // Step 2: Compute Hilbert transform
167 hilbert_signal = filter1(filter_in);
168
169 // Step 3: Envelope detection (magnitude)
170 envelope = envelope_detector(delayed_signal, hilbert_signal);
171 envelope *= gain1;
172
173 // Step 4: Smooth envelope with IIR filter
174 filtered_envelope = applyIIRFilter(envelope);
175 filtered_envelope *= gain2;
176
177 // Step 5: Downsample the smoothed envelope
178 downsampled_output = downsampler(filtered_envelope);
179
180 // Step 6: Compute mean using your existing mean() function
181 mean_val = mean(downsampled_output);
182 mean_val = mean_val * gain3;
183
184 // Step 7: Subtract mean from downsampled output
185 final_output = downsampled_output - mean_val;
186
187 return final_output;
188 //return mean_val;
189 }
```

The testbench is shown below:

```

1  /*
2  * AM Demodulator Testbench
3  *
4  * Description:
5  * This testbench validates an AM (Amplitude Modulation) demodulator implemented
6  * using the Hilbert transform approach. The demodulator processes AM signals
7  * through the following pipeline:
8  *
9  * 1. Delay Line: 16-sample delay to compensate for Hilbert transform group delay
10 * 2. Hilbert Transform: 32-tap FIR filter to generate quadrature component
11 * 3. Envelope Detection: Computes magnitude using  $\sqrt{I^2 + Q^2}$  with
12 *   Newton-Raphson approximation for square root
13 * 4. IIR Smoothing Filter: Lowpass filter to smooth the envelope
14 * 5. Downsampler: Reduces sample rate by factor of 15
15 * 6. DC Removal: Subtracts running mean to remove DC offset
16 *
17 * The testbench generates a standard AM signal:  $s(t) = (1 + m \cdot \cos(2\pi f_m t)) \cdot \cos(2\pi f_c t)$ 
18 * where  $m$  is the modulation index,  $f_m$  is the message frequency, and  $f_c$  is the carrier.
19 *
20 * Test Parameters:
21 * - Sampling Rate: 480 kHz
22 * - Carrier Frequency: 100 kHz
23 * - Message Frequency: 1 kHz (audio tone)
24 * - Modulation Index: 0.8 (80%)
25 *
26 * Output:
27 * - Console output with pass/fail statistics and error metrics
28 * - Data files for visualization and further analysis
29 */
30
31 #include "am_demodulator.hpp"
32 #include <iostream>
33 #include <fstream>
34 #include <cmath>
35 #include <vector>
36
37 #ifndef M_PI
38 #define M_PI 3.14159265358979323846
39 #endif
40
41 #define INITIAL_DELAY 436
42
43 // Test parameters
44 const int NUM_SAMPLES = 180000;
45 const float SAMPLE_RATE = 480000.0; // 480 kHz sampling rate
46 const float CARRIER_FREQ = 100000.0; // 100 kHz carrier
47 const float MESSAGE_FREQ = 1000.0; // 1 kHz message signal

```

```

48 const float CARRIER_FREQ = 100000.0; // 100 kHz carrier
49 const float MESSAGE_FREQ = 1000.0; // 1 kHz message signal
50 const float MODULATION_INDEX = 0.8; // 80% modulation depth
51
52 int main() {
53     std::cout << "===== " << std::endl;
54     std::cout << "AM Demodulator Testbench" << std::endl;
55     std::cout << "===== " << std::endl;
56     std::cout << "Sample Rate: " << SAMPLE_RATE << " Hz" << std::endl;
57     std::cout << "Carrier Frequency: " << CARRIER_FREQ << " Hz" << std::endl;
58     std::cout << "Message Frequency: " << MESSAGE_FREQ << " Hz" << std::endl;
59     std::cout << "Modulation Index: " << MODULATION_INDEX << std::endl;
60     std::cout << "Number of Samples: " << NUM_SAMPLES << std::endl;
61     std::cout << "===== " << std::endl;
62
63     // Buffer to store output samples
64     std::vector<float> output_buffer;
65     output_buffer.reserve(NUM_SAMPLES);
66
67     // Open output files for analysis
68     std::ofstream input_file("am_input.dat");
69     std::ofstream output_file("am_output.dat");
70     std::ofstream message_file("message_signal.dat");
71
72     if (!input_file.is_open() || !output_file.is_open() || !message_file.is_open()) {
73         std::cerr << "Error: Could not open output files!" << std::endl;
74         return -1;
75     }
76
77     // Test counters
78     int test_passed = 0;
79     int test_failed = 0;
80     float max_error = 0.0;
81     float sum_squared_error = 0.0;
82
83     std::cout << "\nGenerating AM signal and processing..." << std::endl;
84
85     // Process samples through the demodulator
86     for (int i = 0; i < NUM_SAMPLES; i++) {
87         float t = (float)i / SAMPLE_RATE;
88
89         // Generate message signal (baseband)
90         float message = cos(2.0 * M_PI * MESSAGE_FREQ * t);
91
92         // Generate AM modulated signal:  $s(t) = (1 + m \cdot \text{message}(t)) \cdot \cos(2\pi f_c t)$ 
93         float am_signal = (1.0 + MODULATION_INDEX * message) * cos(2.0 * M_PI * CARRIER_FREQ * t);

```

```

92
93 // Convert to fixed-point
94 data_type input_sample = am_signal;
95
96 // Run through demodulator
97 data_type output_sample = am_demodulator(input_sample);
98
99 // Convert back to float for analysis
100 float output_float = output_sample.to_float();
101 //float output_float = output_sample;
102
103 // Store output in buffer
104 output_buffer.push_back(output_float);
105
106 // Write input and message to files immediately
107 input_file << i << " " << am_signal << std::endl;
108 message_file << i << " " << message * MODULATION_INDEX << std::endl;
109
110 // Error checking (skip initial transient - first 500 samples)
111 if (i > 500) {
112     float expected = message * MODULATION_INDEX;
113     float error = fabs(output_float - expected);
114
115     if (error > max_error) {
116         max_error = error;
117     }
118     sum_squared_error += error * error;
119
120     // Check if output is within reasonable bounds
121     if (error < 0.3) { // Tolerance for demodulation error
122         test_passed++;
123     } else {
124         test_failed++;
125     }
126 }
127
128 // Progress indicator
129 if ((i + 1) % 1000 == 0) {
130     std::cout << " Processed " << (i + 1) << " samples..." << std::endl;
131 }
132 }
133
134 // Write output with INITIAL_DELAY-sample delay (prepend INITIAL_DELAY zeros, skip last INITIAL_DELAY samples)
135 for (int i = 0; i < INITIAL_DELAY; i++) {
136     output_file << i << " " << 0.0 << std::endl;
137 }

```

```

137 }
138 for (int i = 0; i < NUM_SAMPLES - INITIAL_DELAY; i++) {
139     output_file << (i + INITIAL_DELAY) << " " << output_buffer[i] << std::endl;
140 }
141
142 // Close files
143 input_file.close();
144 output_file.close();
145 message_file.close();
146
147 // Calculate statistics
148 int total_checked = test_passed + test_failed;
149 float mse = sum_squared_error / total_checked;
150 float rmse = sqrt(mse);
151
152 // Print results
153 std::cout << "\n===== " << std::endl;
154 std::cout << "Test Results:" << std::endl;
155 std::cout << "===== " << std::endl;
156 std::cout << "Samples checked: " << total_checked << std::endl;
157 std::cout << "Passed: " << test_passed << " ("
158 << (100.0 * test_passed / total_checked) << "%)" << std::endl;
159 std::cout << "Failed: " << test_failed << " ("
160 << (100.0 * test_failed / total_checked) << "%)" << std::endl;
161 std::cout << "Maximum Error: " << max_error << std::endl;
162 std::cout << "RMSE: " << rmse << std::endl;
163 std::cout << "===== " << std::endl;
164
165 // Determine overall pass/fail
166 bool overall_pass = (test_failed < total_checked * 0.1) && (max_error < 0.5);
167
168 if (overall_pass) {
169     std::cout << "\n*** TEST PASSED ***" << std::endl;
170     std::cout << "The AM demodulator is working correctly!" << std::endl;
171 } else {
172     std::cout << "\n*** TEST FAILED ***" << std::endl;
173     std::cout << "The demodulator output has excessive errors." << std::endl;
174 }
175
176 std::cout << "\nOutput files generated:" << std::endl;
177 std::cout << " - am_input.dat (input AM signal)" << std::endl;
178 std::cout << " - am_output.dat (demodulated output)" << std::endl;
179 std::cout << " - message_signal.dat (expected output)" << std::endl;
180 std::cout << "\nYou can plot these with gnuplot or MATLAB/Python" << std::endl;
181 std::cout << "Example gnuplot command:" << std::endl;
182 std::cout << " plot 'am_output.dat' w l, 'message_signal.dat' w l" << std::endl;
183

```

```

181 std::cout << "Example gnuplot command:" << std::endl;
182 std::cout << " plot 'am_output.dat' w l, 'message_signal.dat' w l" << std::endl;
183
184 // Run Python plotting script
185 system("python3 ./plot_csv.py");
186
187 return overall_pass ? 0 : 1;
188 }
189

```


This testbench validates an AM demodulator implementation using a Hilbert transform-based architecture. It generates a known AM test signal, processes it through the demodulator, and compares the output against the expected baseband message signal to verify correct operation.

The testbench generates a standard AM signal following the mathematical model $s(t) = [1 + m \cdot \cos(2\pi f_m \cdot t)] \cdot \cos(2\pi f_c \cdot t)$, where m is the modulation index, f_m is the message frequency, and f_c is the carrier frequency. The test uses a 480 kHz sampling rate with a 100 kHz carrier frequency and a 1 kHz audio message tone. The modulation index is set to 0.8, representing 80% modulation depth. A total of 180,000 samples are processed, providing approximately 375 milliseconds of signal data.

The demodulator implements a multi-stage signal processing pipeline consisting of a 16-sample delay line to compensate for Hilbert transform group delay, a 32-tap FIR Hilbert transform to generate the quadrature component, envelope detection using magnitude computation with Newton-Raphson square root approximation, an IIR smoothing filter to remove residual carrier components, a downsampler with a factor of 15 to reduce the sample rate to 32 kHz, and DC removal by subtracting the running mean.

For each sample, the testbench calculates the time value as `sample_index` divided by `sample_rate`, generates the baseband message as $\cos(2\pi \cdot 1\text{kHz} \cdot t)$, and creates the AM modulated signal. The signal is converted to the appropriate data type and passed through the `am_demodulator` function. The output is collected and stored in a buffer for later processing.

The demodulator introduces processing delay due to its filtering stages. To align the output with input for comparison, the testbench prepends 29 zero samples to the output data and discards the last 29 samples of demodulator output. This ensures all data files contain exactly 180,000 samples with proper temporal alignment. The 29-sample delay compensation accounts for the Hilbert transform FIR filter group delay and additional processing delays in the envelope detection and smoothing stages.

The testbench excludes the first 500 samples from error analysis to allow filter states to stabilize and avoid startup transients. It computes the absolute difference between output and expected message signal for each sample, tracks the maximum error observed, and calculates both mean squared error and root mean squared error. An individual sample passes if its error is less than 0.3, and the overall test passes if less than 10% of samples fail and the maximum error is below 0.5.

Three data files are generated: `am_input.dat` contains the generated AM modulated input signal, `am_output.dat` contains the demodulated output aligned with the input by

prepending zeros, and `message_signal.dat` contains the expected baseband message signal used as the reference. All files use the format `sample_index` followed by amplitude value, with 180,000 samples each.

The testbench provides real-time progress updates every 1,000 samples and displays a statistical summary including total samples checked, number and percentage of passed and failed samples, maximum error, RMSE value, and an overall pass or fail verdict. After data generation, it automatically invokes a Python plotting script to generate visual representations of the waveforms, frequency spectrum, and error plots.

The error tolerance values are configurable and should be adjusted based on application precision requirements, fixed-point quantization effects, filter approximation errors, and target signal-to-noise ratio. The testbench supports both fixed-point implementations using `to_float()` conversion and floating-point implementations through direct assignment. The exact delay compensation value may need adjustment if filter coefficients or architecture changes occur.

The python script visualizes the performance of a Hilbert transform-based AM demodulator by analyzing output data from the testbench. It provides comprehensive time-domain, frequency-domain, and error analysis to validate the demodulator's functionality and performance.

The script reads three input files generated by the testbench: `am_input.dat` containing the input AM modulated signal, `am_output.dat` containing the demodulated output signal, and `message_signal.dat` containing the expected message signal for comparison. Each file uses the format of index followed by amplitude value.

Three visualization plots are generated and saved as high-resolution PNG images. The time domain analysis plot shows the input AM signal with carrier and envelope modulation, an overlay comparison of demodulated output versus expected message, and the error signal to visualize demodulation accuracy. The frequency domain analysis plot displays the FFT magnitude spectrum showing frequency content and power spectral density using the Welch method for noise analysis, comparing demodulated and expected signals. The error analysis plot presents an error distribution histogram, cumulative absolute error over time, an error versus output scatter plot, and a statistical summary including mean, standard deviation, maximum error, RMSE, and signal-to-noise ratio.

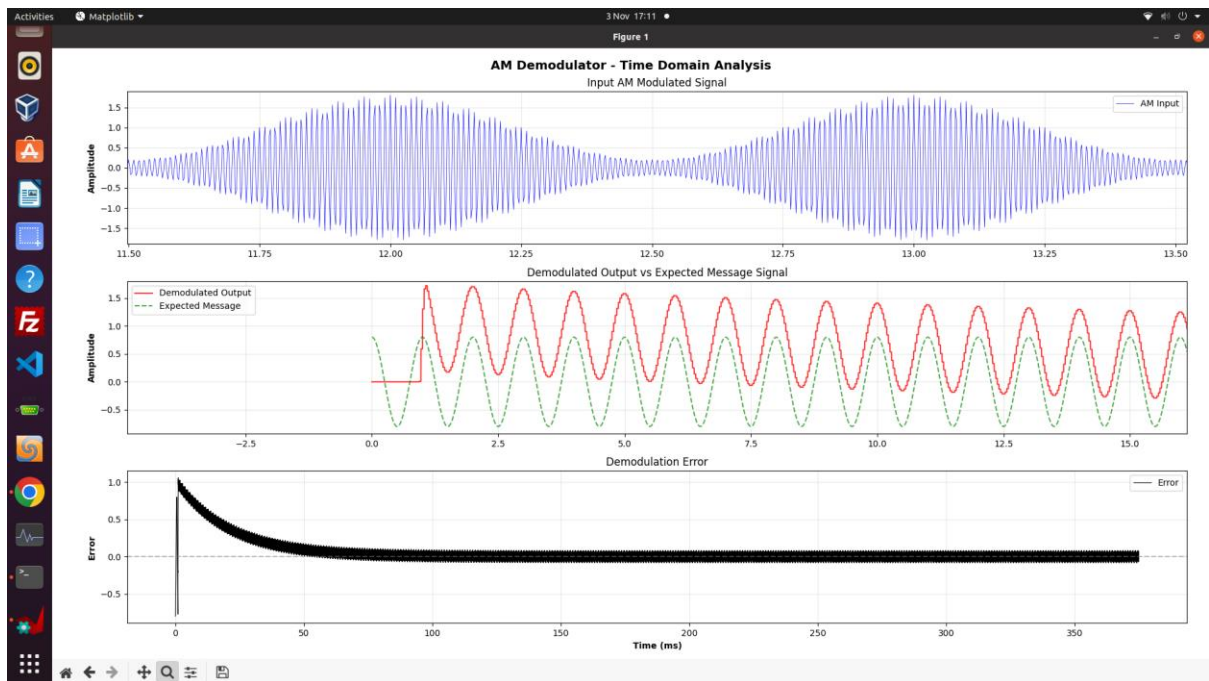
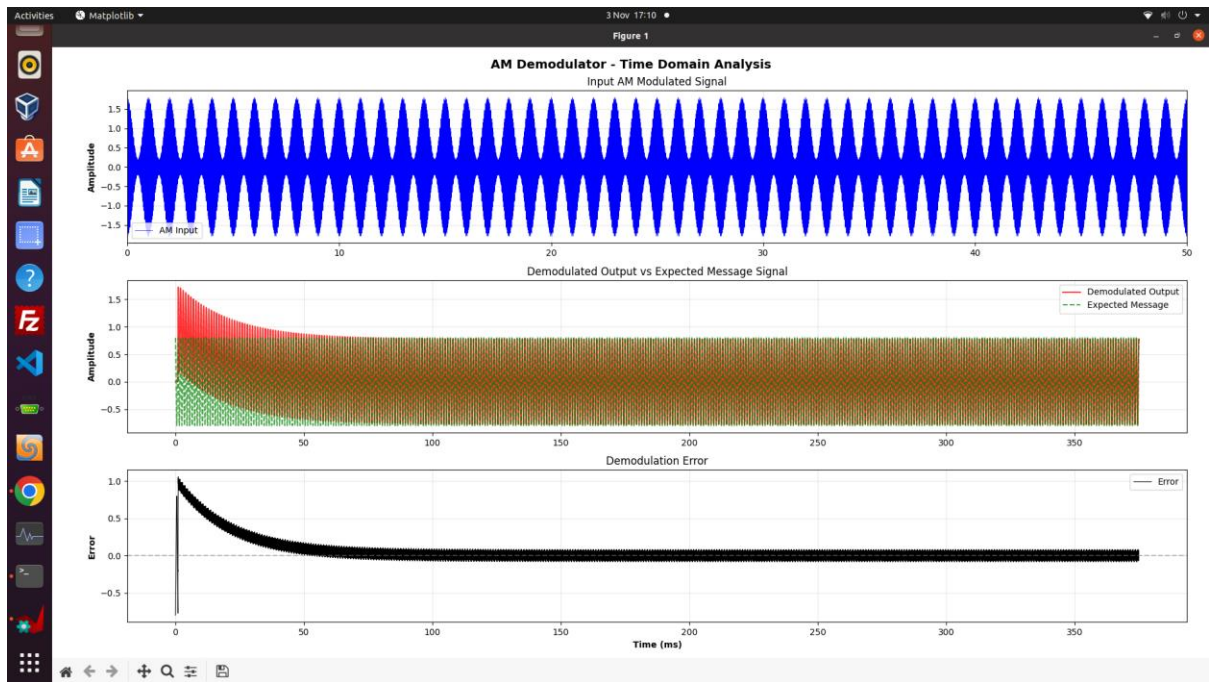
The script automatically validates that all required input files exist before processing and provides helpful error messages if files are missing. It skips the initial 500 samples to avoid filter transient effects when calculating error statistics. The SNR calculation quantifies demodulation quality by computing the ratio of signal power to noise power

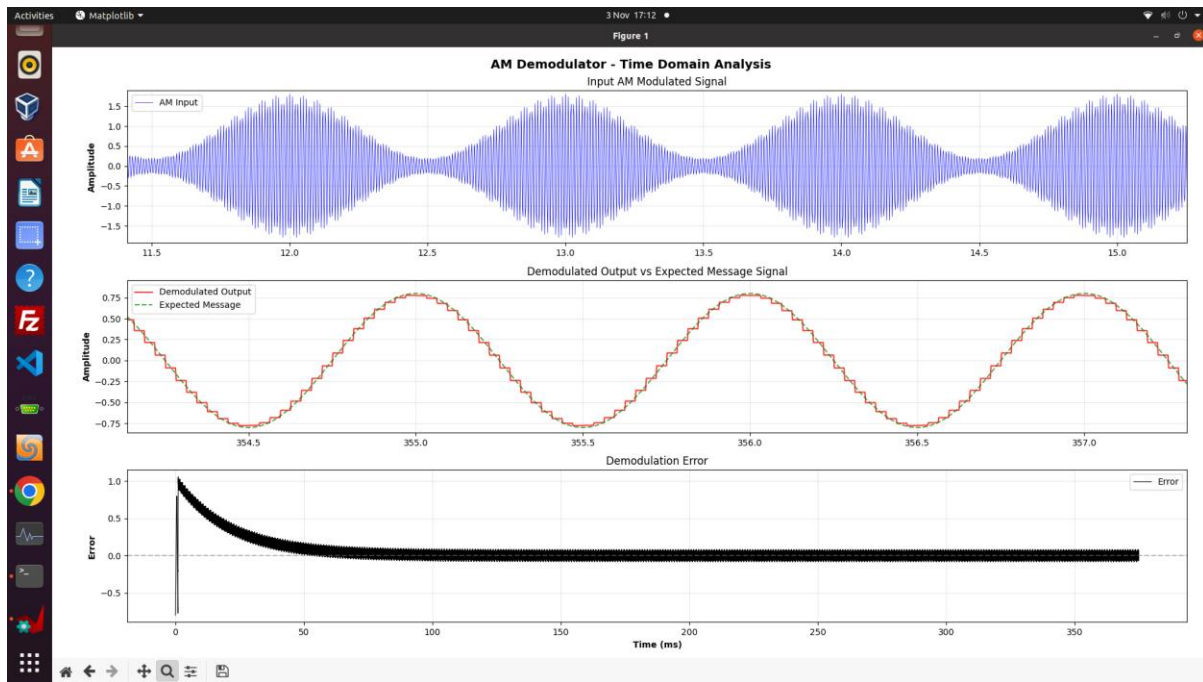
in decibels. All plots feature professional formatting with labeled axes, legends, and grid lines for easy interpretation.

The time domain plot converts sample indices to milliseconds for the time axis and displays the first 50 milliseconds of the input AM signal to clearly show modulation characteristics. The demodulated output is overlaid with the expected message signal to allow direct visual comparison, while the error signal plot includes a zero reference line to identify bias. The frequency domain analysis uses FFT to compute magnitude spectra and displays only positive frequencies up to 10 kHz for clarity. Power spectral density is calculated using Welch's method with 1024-point segments to provide smooth spectral estimates for noise analysis.

Error statistics are computed only on steady-state samples after the 500-sample transient period. The error histogram shows the distribution of demodulation errors with a red vertical line at zero error to indicate bias. The cumulative absolute error plot reveals if errors accumulate over time or remain bounded. The scatter plot of error versus output value helps identify if errors are amplitude-dependent or systematic. Statistical metrics include mean error to detect DC bias, standard deviation to measure error variability, maximum absolute error for worst-case analysis, RMSE for overall accuracy, and SNR computed as $10 \cdot \log_{10}(\text{signal_power}/\text{error_power})$ to quantify demodulation quality.

The script requires Python 3 with NumPy for numerical operations and FFT computation, Matplotlib for plotting and visualization, and SciPy for advanced signal processing functions like Welch PSD estimation. These dependencies can be installed using `pip install numpy matplotlib scipy`. The script should be run in the same directory as the data files generated by the testbench using the command `python3 plot_am_results.py`.





All plots are saved with 300 DPI resolution for publication quality and displayed interactively for immediate review. The script provides progress messages during execution to inform the user of its status and confirms successful completion with checkmarks for each generated plot. If any errors occur during file reading or processing, descriptive error messages guide the user toward resolution.

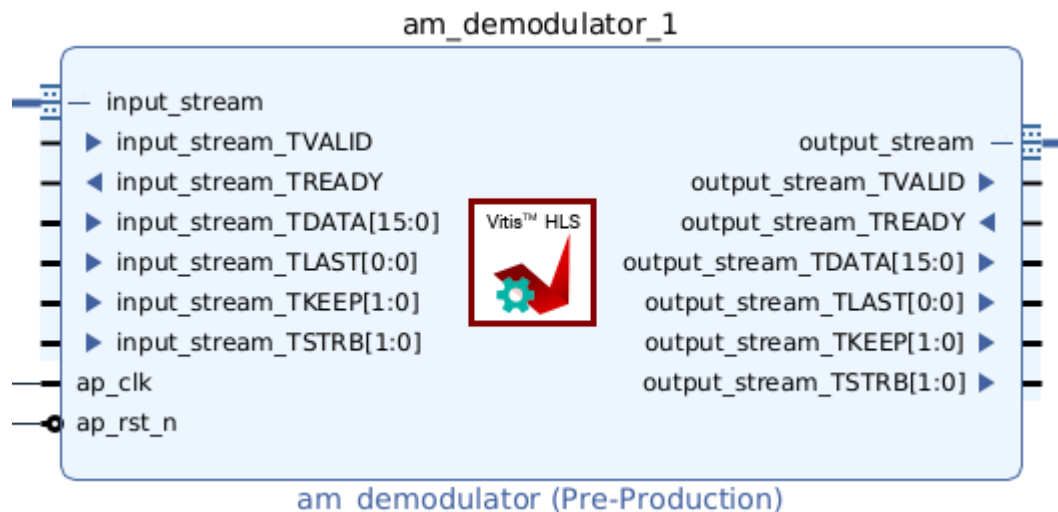
In terms of resource usage we get:

Resource Usage	
	Verilog
SLICE	0
LUT	66101
FF	69393
DSP	12
BRAM	3
URAM	0
LATCH	0
SRL	4194
CLB	0

Final Timing	
	Verilog
CP required	10.000
CP achieved post-synthesis	8.285

Timing met

Importing as an IP-core in Vivado:



We can see it has AXI stream I/Os.

By looking at the synthesis report:

project_1 - /home/caccolillo/hilbert_transform_am/vivado/receiver/project_1/project_1.xpr - Vivado 2022.2

Synthesized Design - synth_1 | xc7a100tcsq324-1

Synthesis Complete ✓

Default Layout

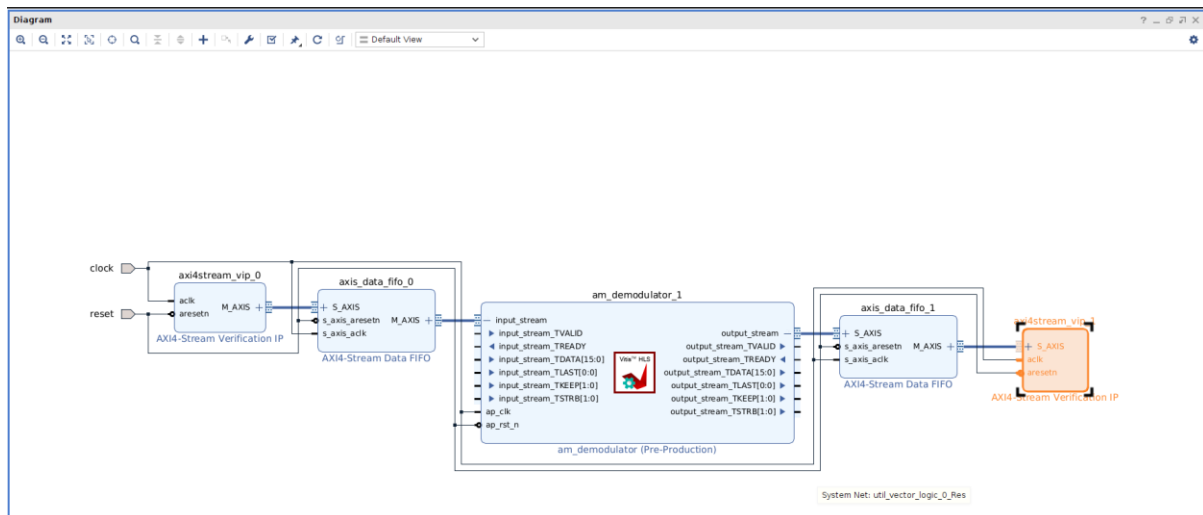
Name	Slice LUTs (63400)	Slice Registers (126800)	DSPs (240)	Bonded IOB (230)
design_1_wrapper	13589	14122	34	48
design_1_0 (design_1)	13589	14122	34	0
am_demodulator_1_0 (design_1_0.am_demodulator_1_0)	13589	14122	34	0
buffer_V_U (design_1_0.am_demodulator_1_0.am_demodulator_buffer_V_RAM_AUTO_1R1W)	16	16	0	0
grp_envelope_detector_fu_709 (design_1_0.am_demodulator_1_0.am_demodulator_envelope_detector)	12228	12507	2	0
mac_muladd_16s_5ns_24s_24_4_1_U47 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_5ns_24s_24_4_1)	25	0	1	0
mac_muladd_16s_6ns_24s_24_4_1_U48 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_6ns_24s_24_4_1)	32	0	2	0
mac_muladd_16s_6s_23s_23_4_1_U45 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_6s_23s_23_4_1)	54	0	2	0
mac_muladd_16s_7s_24s_24_4_1_U44 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_7s_24s_24_4_1)	16	0	2	0
mac_muladd_16s_7s_24s_24_4_1_U36 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_7s_24s_24_4_1)	34	0	2	0
mac_muladd_16s_8ns_26s_26_4_1_U45 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_8ns_26s_26_4_1)	34	0	2	0
mac_muladd_16s_8ns_24s_25_4_1_U31 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_8ns_24s_25_4_1)	37	0	1	0
mac_muladd_16s_9ns_27s_27_4_1_U44 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_9ns_27s_27_4_1)	57	0	2	0
mac_muladd_16s_9s_26s_26_4_1_U37 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_9s_26s_26_4_1)	32	0	2	0
mac_muladd_16s_10ns_28s_28_4_1_U43 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_10ns_28s_28_4_1)	34	0	2	0
mac_muladd_16s_10s_24s_25_4_1_U21 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_10s_24s_25_4_1)	32	0	1	0
mac_muladd_16s_10s_27s_27_4_1_U38 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_10s_27s_27_4_1)	58	0	2	0
mac_muladd_16s_11ns_29s_29_4_1_U42 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_11ns_29s_29_4_1)	59	0	2	0
mac_muladd_16s_11s_28s_28_4_1_U39 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_11s_28s_28_4_1)	34	0	2	0
mac_muladd_16s_12s_29s_29_4_1_U40 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_12s_29s_29_4_1)	60	0	2	0
mac_muladd_16s_15s_31s_31_4_1_U41 (design_1_0.am_demodulator_1_0.am_demodulator_mac_muladd_16s_15s_31s_31_4_1)	35	0	2	0
mean_val_V_mean_fu_715 (design_1_0.am_demodulator_1_0.am_demodulator_mean)	78	16	0	0
mul_mul_16s_23_4_1_U48 (design_1_0.am_demodulator_1_0.am_demodulator_mul_mul_16s_23_4_1)	0	0	1	0
mul_mul_16s_14ns_31_2_1_U50 (design_1_0.am_demodulator_1_0.am_demodulator_mul_mul_16s_14ns_31_2_1)	118	0	1	0
mul_mul_16s_32_2_1_U49 (design_1_0.am_demodulator_1_0.am_demodulator_mul_mul_16s_32_2_1)	17	0	1	0
regslice_both_input_stream_V_data_V_U (design_1_0.am_demodulator_1_0.am_demodulator_regslice_both)	20	36	0	0
regslice_both_input_stream_V_last_V_U (design_1_0.am_demodulator_1_0.am_demodulator_regslice_both_parameterized3)	5	6	0	0
regslice_both_output_stream_V_data_V_U (design_1_0.am_demodulator_1_0.am_demodulator_regslice_both)	56	36	0	0
regslice_both_output_stream_V_last_V_U (design_1_0.am_demodulator_1_0.am_demodulator_regslice_both_parameterized3_2)	6	6	0	0

utilization_1

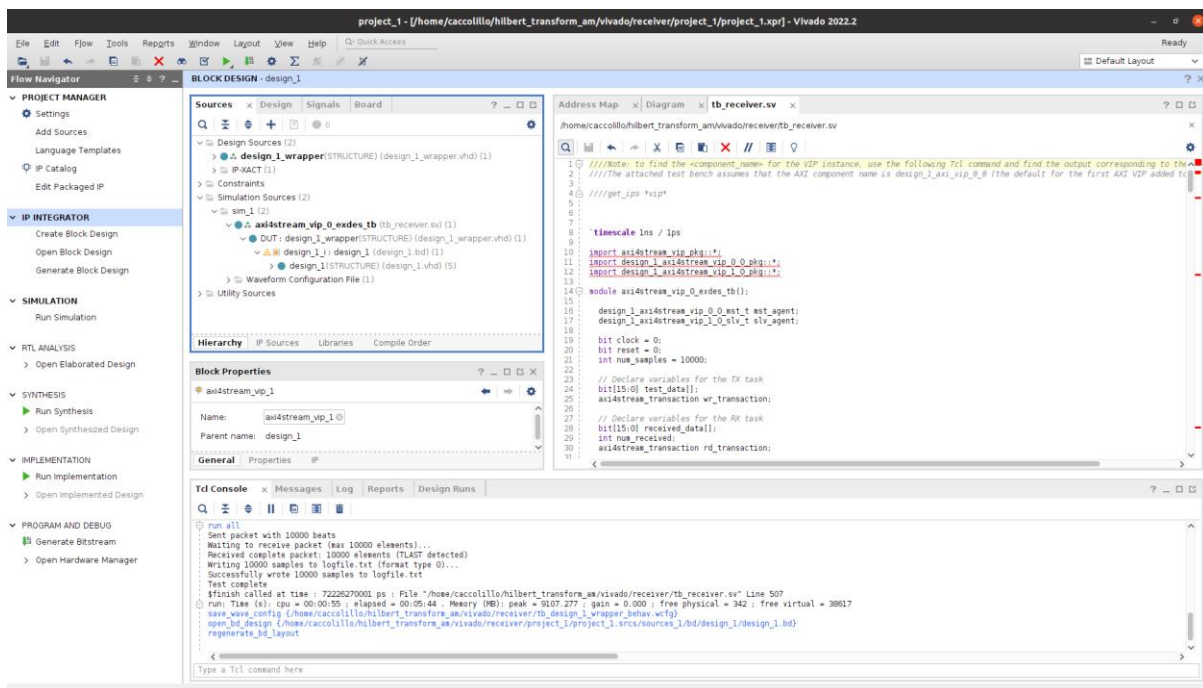
We have an insight into how resources are being used.

THE VIVADO TESTBENCH

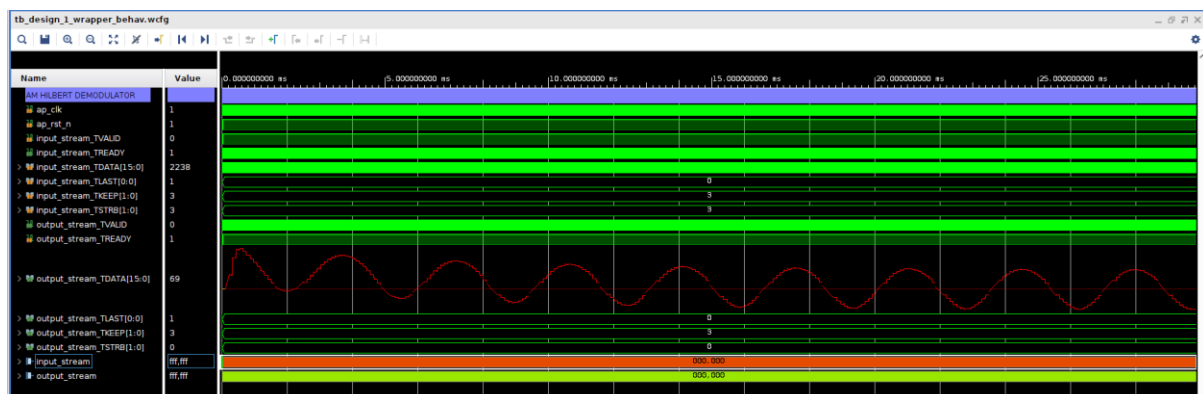
A Vivado design has been created, with the aim to test the IP-core by the means of a couple of AXI Stream VIPs:

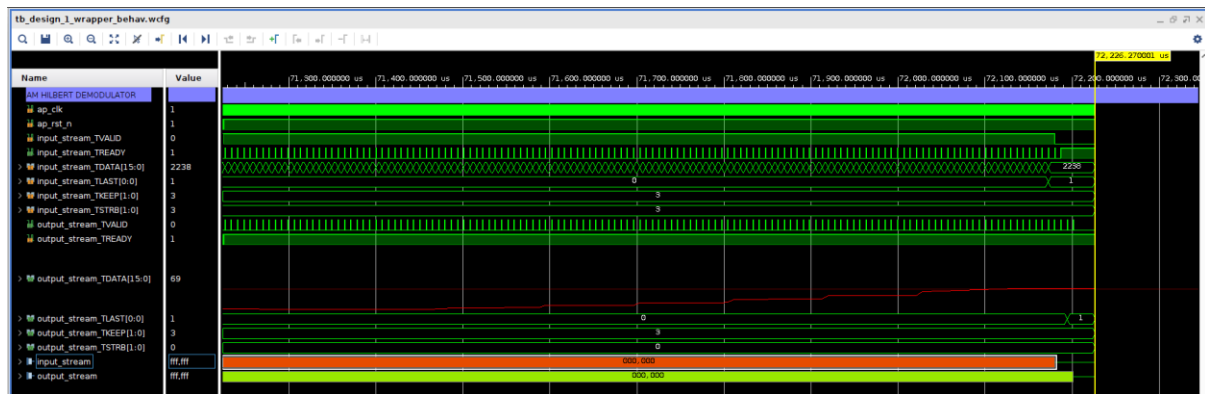


A system verilog testbench is used to inject an AM signal to the DUT, and to collect on a log file the demodulated output:



Key waveforms can be seen by executing the simulation:





This testbench demonstrates an AXI4-Stream verification setup using Xilinx VIP to send and receive fixed-point waveform data through a DUT. It generates an AM-modulated signal in `ap_fixed<16,4>` format, sends it beat-by-beat through the AXI4-Stream Master VIP, and captures the DUT output using the Slave VIP's blocking monitor interface. The master asserts `TLAST` on the final beat, and the slave is configured with no backpressure so it always accepts data. The testbench includes reusable tasks for generating sine and AM signals, transmitting packets, receiving packets until `TLAST`, and saving results to a file in several formats. After initialising clock, reset, and VIP agents, the test generates 10,000 AM samples, streams them into the DUT, captures the response, writes the results to a logfile, and ends the simulation.

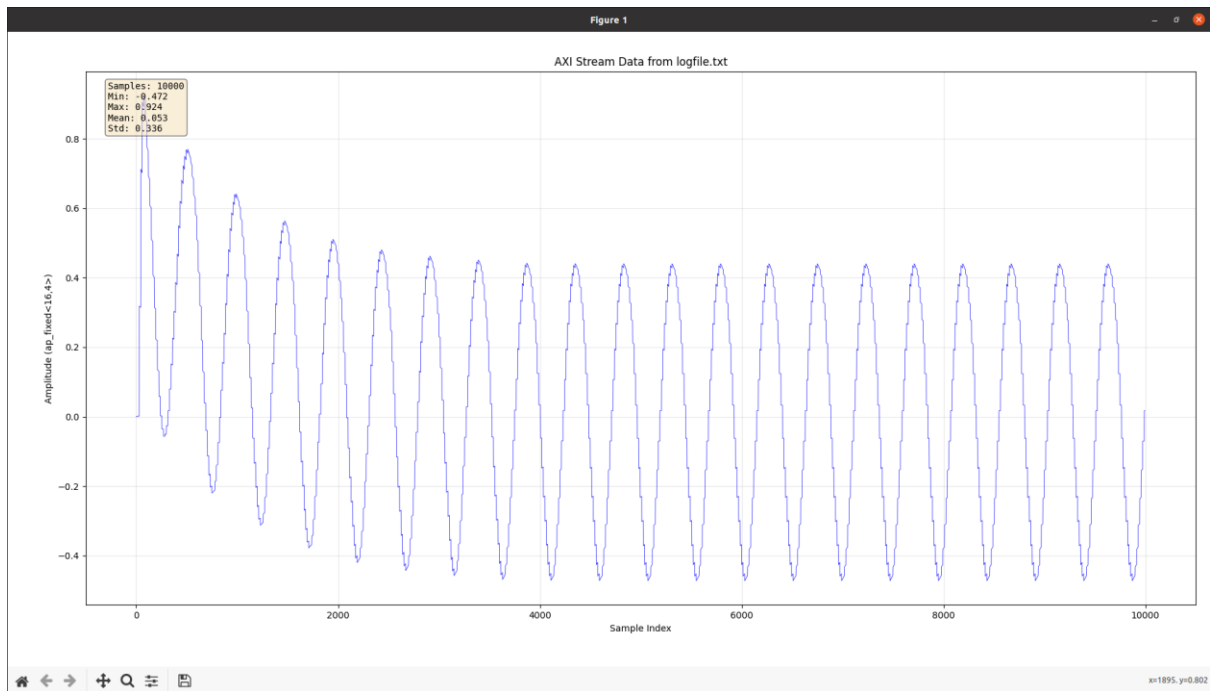
An utility python script has been created to plot the logfile acquired via the AXI stream slave VIP:

```

Open  [ ]  plot.py  ~hibert_transform_analysado/receiver  Save  [ ]  [ ]  [ ]
githubtoken.txt  plot.py

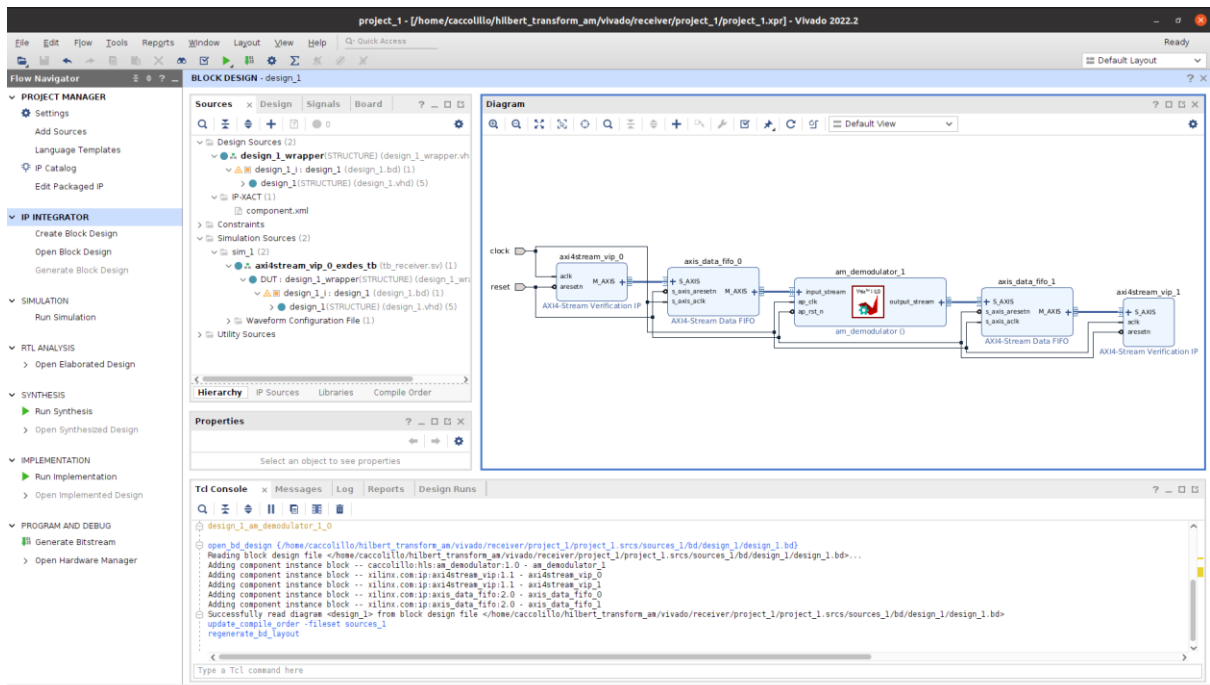
1#!/usr/bin/env python3
2'''
3Plot AXI Stream data from logfile.txt (simple hex format)
4Converts ap_fixed<16,4> format to real values and plots the waveform
5'''
6
7import matplotlib.pyplot as plt
8import numpy as np
9
10def read_hex_file(filename):
11    """
12    Read hex values from file and convert to ap_fixed<16,4> real values
13
14    Args:
15        filename: Path to the hex data file
16
17    Returns:
18        numpy array of real values
19    """
20    data = []
21
22    with open(filename, 'r') as f:
23        for line in f:
24            line = line.strip()
25            if line and not line.startswith('#'):
26                # Read hex value
27                hex_val = int(line, 16)
28
29                # Convert to signed 16-bit integer
30                if hex_val >= 0x8000:
31                    signed_val = hex_val - 0x10000
32                else:
33                    signed_val = hex_val
34
35                # Convert from ap_fixed<16,4> to real (divide by 2^12 = 4096)
36                real_val = signed_val / 4096.0
37                data.append(real_val)
38
39    return np.array(data)
40
41def plot_waveform(data, filename='logfile.txt'):
42    """
43    Plot the waveform data
44
45    Args:
46        data: numpy array of samples
47        filename: name of input file (for title)
48    """
49    plt.figure(figsize=(12, 6))
50
51    # Create sample index
52    samples = np.arange(len(data))
53

```

THE VIVADO TESTBENCH WITH AXI STREAM Verification IP

The block diagram used for the testbench is:



From left to right, we have:

1. AXI4-Stream VIP (Master)
 - a. axi4stream_vip_0

- b. Acts as a source of AXI4-Stream data
 - c. Drives TDATA, TVALID, TLAST, TKEEP
- 2. AXI4-Stream Data FIFO (axis_data_fifo_0)
 - a. Buffers incoming samples
 - b. Decouples timing between VIP and the custom logic
- 3. am_demodulator_1 (HLS IP)
 - a. Custom AM demodulator
 - b. Input stream: input_stream
 - c. Output stream: output_stream
 - d. Data type: ap_fixed<16,4> carried over AXI4-Stream
- 4. AXI4-Stream Data FIFO (axis_data_fifo_1)
 - a. Buffers demodulated samples
- 5. AXI4-Stream VIP (Slave)
 - a. axi4stream_vip_1
 - b. Acts as a sink / monitor
 - c. Captures output data from the demodulator

Clock and reset are shared across the entire chain.

A system Verilog testbench has been prepared:

```

tb_receiver.vv
/home/caccolio/hilbert_transform_am/vvado/receiver/tb_receiver.vv

1 //Note: to find the <component_name> for the VIP instance, use the following Tcl command and find the output corresponding to the AXI VIP instance.
2 //The attached test bench assumes that the AXI component name is design_1_axi_vip_0_0 (the default for the first AXI VIP added to a BO)
3
4 //get_ips *vip*
5
6
7
8
9
10
11
12
13
14 module axi4stream_vip_0_exdes_tb();
15
16     design_1_axi4stream_vip_0_0_ext_t ext_agent;
17     design_1_axi4stream_vip_0_0_ext_t_ext_agent;
18
19     bit clock = 0;
20     bit reset = 0;
21     int num_samples = 10000;
22
23     // Declare variables for the TX task
24     bit[15:0] test_data[];
25     axi4stream_transaction wr_transaction;
26
27     // Declare variables for the RX task
28     bit[15:0] received_data[];
29     int num_received;
30     axi4stream_transaction rd_transaction;
31
32
33     //Instantiate DUT
34     design_1_wrapper DUT(
35         .clock(clock),
36         .reset(reset)
37     );
38
39     //generate clock
40     always #10 clock = ~clock;
41
42
43
44 //*****
45 // Task send_a_packet sends a packet with data from an input array of 16-bit words.
46 // The final 'last' has Tlast set to 1.
47 // This task is used with AXI4STREAM VIP configured to have TLAST and TDATA WIDTH = 16 bits (2 bytes)
48 //*****
49 task automatic send_a_packet;
50     input bit[15:0] data_array[];
51     input int num_elements;
52     output axi4stream_transaction wr_transaction;
53 );
54     bit[2*0-1:0] data_beat;
55     bit[2*1-0] keep_beat; // 2 bits for 2 bytes
56
57 // Validate input
58 if(num_elements <= 0) begin
59     $display("Error: num_elements must be greater than 0");
60     return;
61 end
62

```

The testbench simulates a system with an AXI4-Stream master VIP sending data through a DUT (Device Under Test) to an AXI4-Stream slave VIP.

It's specifically designed for testing signal processing modules that work with 16-bit fixed-point data in ap_fixed<16,4> format (commonly used in Xilinx HLS).

Two VIP Agents are used:

- mst_agent - Master agent that sends data into the DUT
- slv_agent - Slave agent that receives processed data from the DUT
- Both configured for 16-bit TDATA width with TLAST support

The main tasks used in the testbench are:

1. send_a_packet Transmits an array of 16-bit samples via AXI4-Stream, setting TKEEP=0b11 (both bytes valid) and asserting TLAST on the final beat.
2. generate_sine_wave Creates sine wave test data in ap_fixed<16,4> format (1 sign bit, 3 integer bits, 12 fractional bits). Parameters include amplitude, normalized frequency, and sample count.
3. generate_am_signal Generates an AM (amplitude modulated) signal matching a C++ reference testbench. The formula is: $(1 + \text{modulation_index} * \text{message}) * \text{carrier}$ * normalization, where both message and carrier are cosine waves.
4. receive_packet_blocking Blocking read from the slave monitor that collects output data until TLAST is detected or a maximum element count is reached.
5. save_data_to_file Writes received data to a file in three format options: simple hex, detailed with headers, or CSV format suitable for MATLAB/Excel analysis.

The testbench executes this sequence:

1. Initializes and creates VIP agents
2. Releases reset and start agents
3. Configures slave for no backpressure (always ready)
4. Generates an AM signal with 10,000 samples (480kHz sample rate, 100kHz carrier, 1kHz message, 0.6 modulation index)
5. Sends the signal through the DUT
6. Receives the processed output
7. Saves results to "logfile.txt" in simple hex format

THE VIVADO TESTBENCH WITH Zynq UltraScale+ MPSoC Verification IP

The previous testbench uses AXI4-Stream VIP for isolated IP verification. Moving to Zynq UltraScale+ VIP enables comprehensive system-level testing that better represents production deployment.

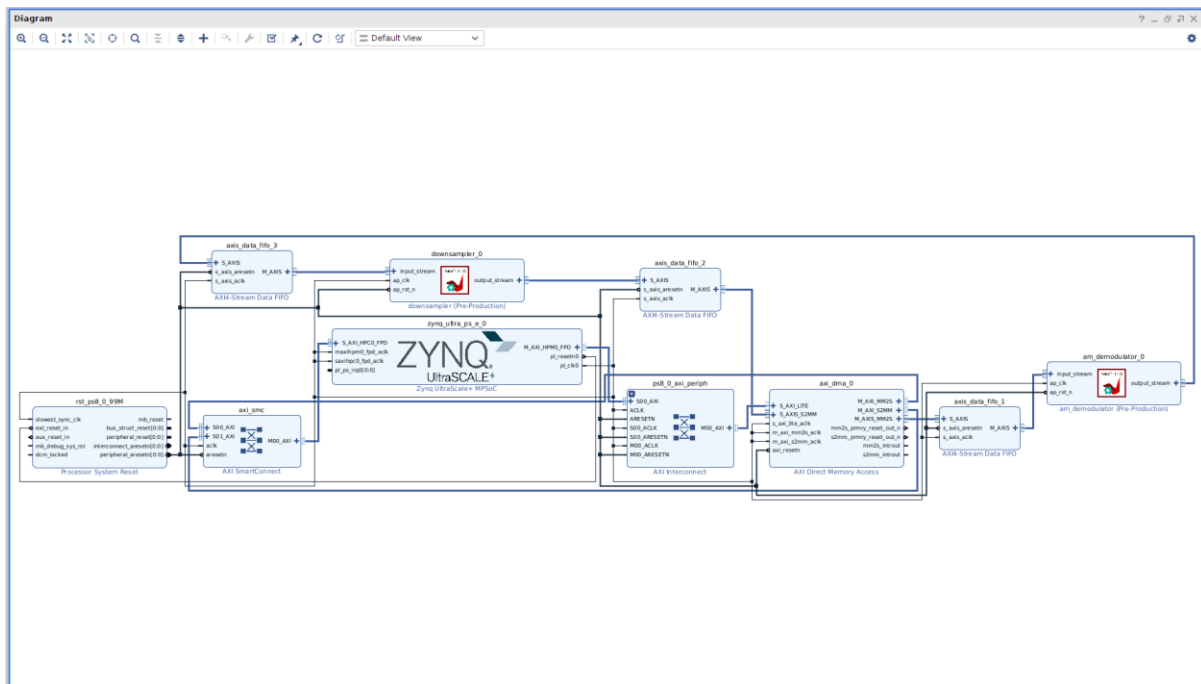
The Zynq UltraScale+ VIP provides a complete processor subsystem including ARM Cortex-A53/R5 cores, PS-PL interfaces, and memory controllers. This validates real-world data paths through HP/HPC/ACP ports and catches integration issues like bus arbitration conflicts and memory bandwidth constraints that isolated testing cannot reveal.

Embedded software can run on simulated ARM cores while interacting with custom hardware. This enables driver development and debugging before hardware availability, verifying register access, interrupt handling, and DMA operations in simulation. Integration issues are caught early when they're easier to fix.

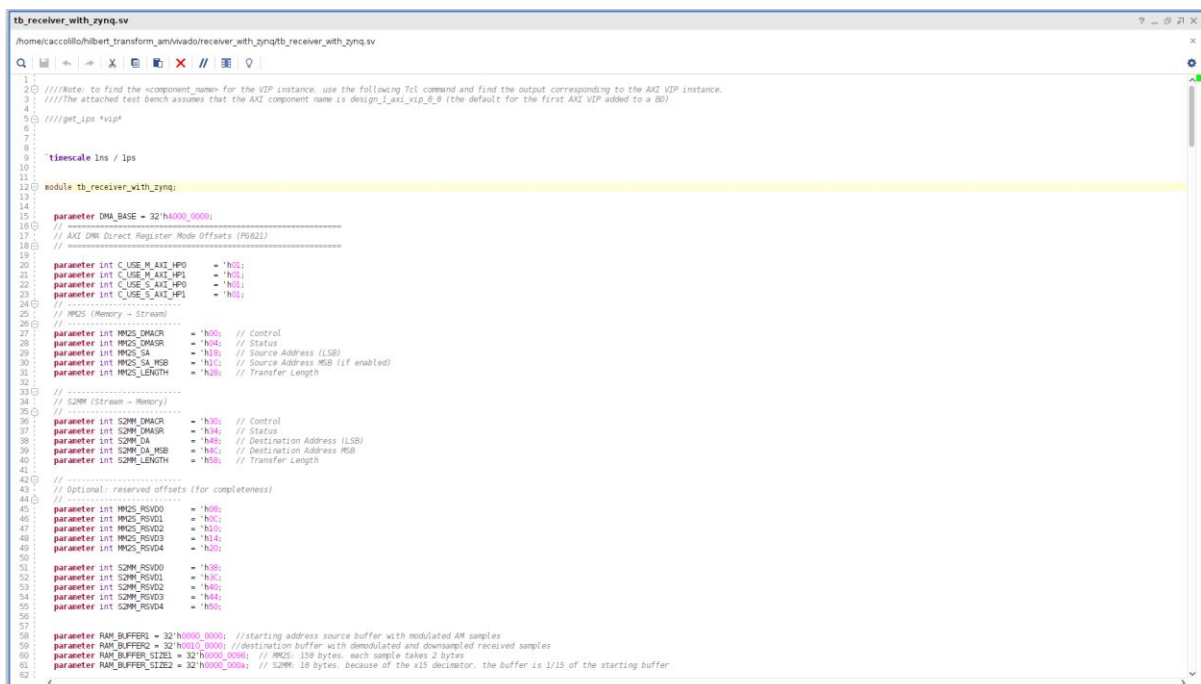
The VIP supports sophisticated scenarios including DMA transfers, cache coherency testing, multi-master arbitration, and realistic memory behavior. End-to-end performance analysis provides accurate predictions of actual hardware operation rather than idealized point-to-point measurements.

Testing the complete operational flow—ARM processor configuration, DMA transfers, signal processing, and result readback—ensures all system aspects work correctly together. Development continues independently of hardware availability, and verification results reliably predict silicon behaviour.

The following block diagram has been used to validate further the architecture in use, so to put the ARM cores into the equation:



Also in this case a system verilog testbench has been written:



This is a SystemVerilog testbench that demonstrates system-level verification using the Zynq UltraScale+ VIP to test an AM demodulator design with integrated DMA transfers.

The testbench instantiates a complete Zynq MPSoC wrapper that includes the ARM processor subsystem VIP, AXI DMA controller, and custom signal processing logic in the programmable logic. Unlike the previous standalone AXI Stream VIP testbench, this environment validates the entire data flow through a realistic embedded system architecture.

The core functionality revolves around DMA-based data movement. The testbench defines two memory buffers in DDR: RAM_BUFFER1 at address 0x00000000 stores the transmitted AM modulated signal with 150 samples per chunk, while RAM_BUFFER2 at address 0x00100000 receives the demodulated and decimated output with only 10 samples per chunk due to a 15x decimation factor in the processing chain.

The DMA control tasks implement the standard Xilinx AXI DMA programming sequence for Direct Register Mode operation. Register offsets are parameterized following the PG021 product guide, with separate control paths for MM2S (Memory to Stream) and S2MM (Stream to Memory) channels. Each channel has control registers, status registers, address pointers, and length registers at defined offsets from the DMA base address of 0xA0000000.

The MM2S programming sequence first resets the channel, then halts it, sets the source address in DDR, enables the channel with run/stop bit set and interrupts masked, and finally writes the transfer length to initiate the operation. The S2MM sequence follows a similar pattern but configures the destination address instead. Polling tasks wait for the idle bit in each channel's status register to confirm transfer completion.

The test flow begins with Zynq VIP initialization, which applies proper reset sequences to both the processing system and programmable logic. The initialization configures the VIP for debug output and sets up slave port profiles for the HP0 and HP1 high-performance AXI interfaces that connect the DMA to DDR memory.

After initialization, the testbench generates 50,000 samples of an AM modulated signal using parameters that match a reference C++ implementation: 480kHz sample rate, 100kHz carrier frequency, 1kHz message frequency, and 0.8 modulation index. The samples are formatted as ap_fixed<16,4> fixed-point values with 4 integer bits and 12 fractional bits, scaled by 4096 for the fractional representation.

The generated AM signal is written to DDR memory using a custom task that packs two 16-bit samples into each 32-bit word to satisfy the Zynq VIP's requirement for 32-bit aligned memory accesses. This packing handles both even and odd sample counts by padding the final word with zeros if necessary.

The main test executes 100 iterations of a DMA transfer loop. Each iteration transmits a 150-byte chunk (75 samples) from RAM_BUFFER1 through the MM2S channel, processes it through the custom demodulator logic, and receives the decimated 10-byte output (5 samples) back to RAM_BUFFER2 via the S2MM channel. Status registers are read before and after each transfer to monitor DMA operation, and clock cycle delays separate the MM2S and S2MM operations to ensure proper sequencing.

After completing all transfers, the testbench uses the Zynq VIP's peek_mem_to_file method to dump both memory buffers to text files for offline analysis. The transmitted buffer and received buffer contents are saved separately, allowing comparison between input and output data to verify correct demodulation and decimation behavior.

This testbench represents a significant step up from simple AXI Stream verification because it validates the complete system integration including processor memory interface timing, DMA register programming, memory bandwidth management, and proper handshaking between multiple AXI interfaces. The test confirms that data can flow from DDR through the DMA, be processed by custom logic, and return to DDR with correct addressing and synchronization, which are the exact operations that will occur in the deployed embedded system.

THE PETALINUX DISTRO

A Petalinux distribution has been prepared

DMA device drivers, Python and USB Ethernet dongle support have been enabled:

```
GTKTerm - /dev/ttyUSB1 115200-8-N-1
File Edit Log Configuration Controls/Signals View Help
[ 9.887460] Bluetooth: RFCOMM TTY layer initialized
[ 9.892249] Bluetooth: RFCOMM socket layer initialized
[ 9.897359] Bluetooth: RFCOMM ver 1.11
[ 9.901866] Bluetooth: BNEP (Ethernet Emulation) ver 1.3
[ 9.906335] Bluetooth: BNEP filters: protocol multicast
[ 9.911530] Bluetooth: BNEP socket layer initialized
[ 9.916455] Bluetooth: HIDP (Human Interface Emulation) ver 1.2
[ 9.922347] Bluetooth: HIDP socket layer initialized
[ 9.927388] 8021q: 802.1Q VLAN Support v1.8
[ 9.931583] 9pnet: Installing 9P2000 support
[ 9.935698] Key type dns_resolver registered
[ 9.940130] registered taskstats version 1
[ 9.943967] Loading compiled-in X.509 certificates
[ 9.950649] Btrfs loaded, crc32c-crc32c-generic, zoned=no, fsverity=no
[ 9.968152] ttf000000.serial: ttyPS1 at MMIO 0xffff000000 (irq = 60, base baud = 6249999) is a xuartps
[ 9.972232] ttf010000.serial: ttyPS0 at MMIO 0xffff010000 (irq = 61, base baud = 6249999) is a xuartps
[ 9.986205] printk: console [ttyPS0] enabled
[ 9.986205] printk: console [ttyPS0] enabled
[ 9.990510] printk: bootconsole [cdns0] disabled
[ 9.990510] printk: bootconsole [cdns0] disabled
[ 9.999736] of-fpga-region fpga-full: FPGA Region probed
[ 10.011380] xilinx-vdma a0000000.dma: error -EINVAL: failed to get irq
[ 10.017926] xilinx-vdma: probe of a0000000.dma failed with error -22
[ 10.024845] xilinx-zynqmp-dma fd500000.dma-controller: ZynqMP DMA driver Probe success
[ 10.032963] xilinx-zynqmp-dma fd510000.dma-controller: ZynqMP DMA driver Probe success
[ 10.041883] xilinx-zynqmp-dma fd520000.dma-controller: ZynqMP DMA driver Probe success
[ 10.049197] xilinx-zynqmp-dma fd530000.dma-controller: ZynqMP DMA driver Probe success
[ 10.057304] xilinx-zynqmp-dma fd540000.dma-controller: ZynqMP DMA driver Probe success
[ 10.065409] xilinx-zynqmp-dma fd550000.dma-controller: ZynqMP DMA driver Probe success
[ 10.073528] xilinx-zynqmp-dma fd560000.dma-controller: ZynqMP DMA driver Probe success
[ 10.081623] xilinx-zynqmp-dma fd570000.dma-controller: ZynqMP DMA driver Probe success
[ 10.089804] xilinx-zynqmp-dma ffa80000.dma-controller: ZynqMP DMA driver Probe success
[ 10.097918] xilinx-zynqmp-dma ffa90000.dma-controller: ZynqMP DMA driver Probe success
[ 10.106024] xilinx-zynqmp-dma ffaa0000.dma-controller: ZynqMP DMA driver Probe success
[ 10.114148] xilinx-zynqmp-dma ffab0000.dma-controller: ZynqMP DMA driver Probe success
[ 10.122251] xilinx-zynqmp-dma ffac0000.dma-controller: ZynqMP DMA driver Probe success
[ 10.130359] xilinx-zynqmp-dma ffad0000.dma-controller: ZynqMP DMA driver Probe success
[ 10.138461] xilinx-zynqmp-dma ffae0000.dma-controller: ZynqMP DMA driver Probe success
[ 10.146574] xilinx-zynqmp-dma ffa00000.dma-controller: ZynqMP DMA driver Probe success
[ 10.154955] xilinx-zynqmp-dpdma fd4c0000.dma-controller: Xilinx DPDMA engine is probed
[ 10.163082] xilinx-psgtr fd400000.phy: Invalid reference clock number 1
[ 10.169695] zynqmp-display fd4a0000.display: failed to get PHY lane 0
[ 10.176168] zynqmp-display: probe of fd4a0000.display failed with error -22
[ 10.184581] xilinx-axiipmon ffa00000.perf-monitor: Probed Xilinx APM
[ 10.191137] xilinx-axiipmon fd0b0000.perf-monitor: Probed Xilinx APM
/dev/ttyUSB1 115200-8-N-1 DTR RTS CTS CD DSR RI
```

```
GTKTerm - /dev/ttyUSB1 115200-8-N-1
File Edit Log Configuration Controlsignals View Help

Login incorrect
ultra96 login:
Login ti
PetaLinux 2022.2_release_S10071807 ultra96 ttyPS0

ultra96 login: admin
Password:

Login incorrect
ultra96 login: petalinux
You are required to change your password immediately (administrator enforced).
New password:
Retype new password:
ultra96:~$
ultra96:~$ lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 003 Device 003: ID 0424:2740 Microchip Technology, Inc. (formerly SMSC) Hub Controller
Bus 003 Device 004: ID 0b95:1790 ASIX Electronics Corp. AX88179 Gigabit Ethernet
Bus 003 Device 002: ID 0424:2744 Microchip Technology, Inc. (formerly SMSC) Hub
Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
ultra96:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 7C:C2:C6:41:8D:D0
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:86 errors:0 dropped:0 overruns:0 frame:0
          TX packets:86 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:6480 (6.3 KiB)  TX bytes:6480 (6.3 KiB)

ultra96:~$ python
Python 3.9.9 (main, Nov 15 2021, 18:05:17)
[GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>>
```

```
GTKTerm - /dev/ttyUSB1 115200-8-N-1
File Edit Log Configuration Controlsignals View Help

[ OK ] Finished Record Runlevel Change in UTMP.

PetaLinux 2022.2_release_S10071807 ultra96 ttyPS0

ultra96 login: petalinux
You are required to change your password immediately (administrator enforced).
New password:
Retype new password:
ultra96:~$ ls /dev
autofs      null      ptybf     ptye2     ptyq5     pty8      ptyub     ptywe     ptyz1     rtc0      tty34     tty9      ttybc     ttydf     ttyq2     ttys5     ttyu8     ttywb     ttyye     vcs4
block       port      ptyc9     ptye3     ptyq6     pty9      ptyuc     ptywf     ptyz2     shm       tty35     ttyPS0    ttybd     ttye0     ttyq3     ttys6     ttyu9     ttywc     ttyyf     vcs5
btrfs-control ptmx     ptyc1     ptye4     ptyq7     ptysa     ptyud     ptyx0     ptyz3     snd       tty36     ttyPS1    ttybe     ttye1     ttyq4     ttys7     ttyua     ttywd     ttyz0     vcs6
bus          pts       ptyc2     ptye5     ptyq8     ptysb     ptyue     ptyx1     ptyz4     stderr    tty37     ttyS0     ttybf     ttye2     ttyq5     ttys8     ttyub     ttywe     ttyz1     vcsa
char         ptya0     ptyc3     ptye6     ptyq9     ptysc     ptyuf     ptyx2     ptyz5     stdin     tty38     ttyS1     ttyc0     ttye3     ttyq6     ttys9     ttyuc     ttywf     ttyz2     vcsa1
console     ptya1     ptyc4     ptye7     ptyqa     ptyvd     ptyv0     ptyx3     ptyz6     stdout    tty39     ttyS2     ttyc1     ttye4     ttyq7     ttysa     ttyud     ttyx0     ttyz3     vcsa2
disk        ptya2     ptyc5     ptye8     ptyqb     ptyse     ptyv1     ptyx4     ptyz7     tty       tty4     ttyS3     ttyc2     ttye5     ttyq8     ttysb     ttyue     ttyx1     ttyz4     vcsa3
dma_heap    ptya3     ptyc6     ptye9     ptyqc     ptyvf     ptyv2     ptyx5     ptyz8     tty0      tty40     ttya0     ttyc3     ttye6     ttyq9     ttysc     ttyuf     ttyx2     ttyz5     vcsa4
fd           ptya4     ptyc7     ptyea     ptyqd     ptyt0     ptyv3     ptyx6     ptyz9     tty1      tty41     ttya1     ttyc5     ttye7     ttya     ttysd     ttyv0     ttyx3     ttyz6     vcsa5
fpga0       ptya5     ptyc8     ptyeb     ptyqe     ptyt1     ptyv4     ptyx7     ptyza     tty10     tty42     ttya2     ttyc5     ttye8     ttyb     ttysf     ttyv1     ttyx4     ttyz7     vcsa6
full        ptya6     ptyc9     ptyec     ptyqf     ptyt2     ptyv5     ptyx8     ptyzb     tty11     tty43     ttya3     ttyc6     ttye9     ttyqc     ttysf     ttyv2     ttyx5     ttyz8     vcsu
gpiochip0   ptya7     ptyca     ptyed     ptyr0     ptyt3     ptyv6     ptyx9     ptyzc     tty12     tty44     ttya4     ttyc7     ttyea     ttyqd     ttyf0     ttyv3     ttyx6     ttyz9     vcsu1
gpiochip1   ptya8     ptycb     ptyee     ptyr1     ptyt4     ptyv7     ptyxa     ptyzd     tty13     tty45     ttya5     ttyc8     ttyeb     ttyqe     ttyt1     ttyv4     ttyx7     ttyza     vcsu2
hugepages   ptya9     ptycc     ptyef     ptyr2     ptyt5     ptyv8     ptyxb     ptyze     tty14     tty46     ttya6     ttyc9     ttyec     ttyqf     ttyt2     ttyv5     ttyx8     ttyzb     vcsu3
i2c-0       ptyaa     ptycd     ptyp0     ptyr3     ptyt6     ptyv9     ptyxc     ptyzf     tty15     tty47     ttya7     ttyca     ttyed     ttyr0     ttyt3     ttyv6     ttyx9     ttyzc     vcsu4
iio:device0 ptyab     ptyce     ptypl     ptyr4     ptyt7     ptyva     ptyxd     ram0     tty16     tty48     ttya8     ttycb     ttyee     ttyr1     ttyt4     ttyv7     ttyxa     ttyzd     vcsu5
initctl     ptyac     ptycf     ptyp2     ptyr5     ptyt8     ptyvb     ptyxe     ram1     tty17     tty49     ttya9     ttycc     ttyef     ttyr2     ttyt5     ttyv8     ttyxb     ttyze     vcsu6
kmsg        ptyad     ptydb     ptyp3     ptyr6     ptyt9     ptyvc     ptyxf     ram10     tty18     tty5     ttyaa     ttycd     ttypp0     ttyr3     ttyt6     ttyv9     ttyxc     ttyzf     vhc1
log          ptyae     ptydl     ptyp4     ptyr7     ptyta     ptyvd     ptyy0     ram11     tty19     tty55     ttyab     ttyce     ttypl     ttyr4     ttyt7     ttyva     ttyxd     ubi_ctrl watchdog
loop-control ptyaf     ptyd2     ptyp5     ptyr8     ptytb     ptyve     ptyy1     ram12     tty2     tty51     ttyac     ttycf     ttyp2     ttyr5     ttyt8     ttyvb     ttyxe     udev_network_queue watchdog0
loop0        ptyb0     ptyd3     ptyp6     ptyr9     ptytc     ptyvf     ptyy2     ram13     tty20     tty52     ttyad     ttyd0     ttyp3     ttyr6     ttyt9     ttyvc     ttyxf     udmabuf watchdog1
loop1        ptyb1     ptyd4     ptyp7     ptyra     ptytd     ptyw0     ptyy3     ram14     tty21     tty53     ttyae     ttyd1     ttyp4     ttyr7     ttyta     ttyvd     ttyy0     ui0 zero
loop2        ptyb2     ptyd5     ptyp8     ptyrb     ptyte     ptyw1     ptyy4     ram15     tty22     tty54     ttyaf     ttyd2     ttyp5     ttyr8     ttytb     ttyve     ttyy1     ui01
loop3        ptyb3     ptyd6     ptyp9     ptyrc     ptytf     ptyw2     ptyy5     ram2     tty23     tty55     ttyb0     ttyd3     ttyp6     ttyr9     ttytc     ttyvf     ttyy2     ui02
loop4        ptyb4     ptyd7     ptypa     ptyrd     ptyu0     ptyy3     ptyy6     ram3     tty24     tty56     ttyb1     ttyd4     ttyp7     ttyra     ttytd     ptyw0     ptyy3     ui03
loop5        ptyb5     ptyd8     ptypb     ptyre     ptyu1     ptyy4     ptyy7     ram4     tty25     tty57     ttyb2     ttyd5     ttyp8     ttyrb     ttyte     ttyy1     ptyw4     urandom
loop6        ptyb6     ptyd9     ptypc     ptyrf     ptyu2     ptyy5     ptyy8     ram5     tty26     tty58     ttyb3     ttyd6     ttyp9     ttyrc     ttytf     ttyy2     ptyw5     usbmon0
loop7        ptyb7     ptyda     ptypd     ptyso     ptyu3     ptyy6     ptyy9     ram6     tty27     tty59     ttyb4     ttyd7     ttypa     ttyrd     ttyy0     ptyw3     usbmon1
mali         ptyb8     ptydb     ptype     ptyu4     ptyw7     ptyya     ram7     tty28     tty6     ttyb5     ttyd8     ttypb     ttyre     ttyu1     ptyw4     ttyy7     usbmon2
mem          ptyb9     ptydc     ptypf     ptyu5     ptyw8     ptyyb     ram8     tty29     tty60     ttyb6     ttyd9     ttypc     ttyrf     ttyu2     ptyw5     ttyy8     usbmon3
mmcblk0      ptyba     ptydd     ptyq0     ptyy3     ptyu6     ptyy9     ptyyc     ram9     tty3     tty61     ttyb7     ttyda     ttypd     ttyy0     ptyw3     ttyy6     usbmon4
mmcblk0p1    ptybb     ptyde     ptyq1     ptyu4     ptyu7     ptyya     ptyyd     random   tty30     tty62     ttyb8     ttydb     ttype     ttyy1     ptyw4     ttyy7     vcs
mmcblk0p2    ptybc     ptydf     ptyq2     ptyu5     ptyu8     ptywb     ptyy6     raw-gadget tty31     tty63     ttyb9     ttydc     ttypf     ttyy2     ptyw5     ttyy8     vcs1
mqueue       ptybd     ptye0     ptyq3     ptyu6     ptyu9     ptywc     ptyyf     rkill    tty32     tty7     ttyba     ttydd     ttyq0     ttyy3     ptyw6     ttyy9     vcs2
net          ptybe     ptyel     ptyq4     ptyy7     ptyua     ptywd     ptyz0     rtc      tty33     tty8     ttybb     ttyde     ttyql     ttyy4     ttyu7     ttyya     ttyyd     vcs3

ultra96:~$
```

a device named u-dma-buf can be seen, so the distribution is ready to be used.

As ethernet will be needed with a static IP address, we can set it on the Ultra96 v2 board as in the following picture:


```
GTKTerm - /dev/ttyUSB1 115200-8-N-1
File Edit Log Configuration Controlsignals View Help
[ 37.875218] audit: type=1006 audit(1637342400.924:2): pid=596 uid=0 old-auid=4294967295 auid=1000 tty=(none) old-ses=4294967295 ses=1 res=1
[ 37.887928] audit: type=1300 audit(1637342400.924:2): arch=c00000b7 syscall=64 success=yes exit=4 a0=8 a1=ffff569f940 a2=4 a3=ffff9a3d96b0 items=
0 ppid=1 pid=596 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=1 comm="(systemd)" exe="/lib/systemd/systemd" key=(
null)
[ 37.914545] audit: type=1327 audit(1637342400.924:2): proctitle="(systemd)"
ultra96:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 7C:C6:41:8D:D0
          inet6 addr: fe80::7ec2:c6ff:fe41:8dd0/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:4 errors:4 dropped:0 overruns:0 frame:0
          TX packets:18 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:184 (184.0 B)  TX bytes:2611 (2.5 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:2 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:140 (140.0 B)  TX bytes:140 (140.0 B)

ultra96:~$ sudo ifconfig eth0 192.168.1.11

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

Password:
ultra96:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 7C:C6:41:8D:D0
          inet addr:192.168.1.11  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::7ec2:c6ff:fe41:8dd0/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:8 errors:8 dropped:0 overruns:0 frame:0
          TX packets:27 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:368 (368.0 B)  TX bytes:3956 (3.8 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:2 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:140 (140.0 B)  TX bytes:140 (140.0 B)
```

/dev/ttyUSB1 115200-8-N-1

DTR RTS CTS CD DSR RI

And on the host Linux PC we do something similar:

```
caccolillo@caccolillo-OMEN-25L-Desktop-GT12-1xxx: ~
INFO: Checking installed development libraries
INFO: Checking network and other services
WARNING: No tftp server found - please refer to "UG1144 2022.2 PetaLinux Tools Documentation Reference Guide" for its impact and solution
caccolillo@caccolillo-OMEN-25L-Desktop-GT12-1xxx:~$ ifconfig
enp5s0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 6c:02:e0:45:16:fb txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 180 bytes 14948 (14.9 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 180 bytes 14948 (14.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlp4s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.149.17 netmask 255.255.255.0 broadcast 192.168.149.255
    inet6 fe80::2c71:d382:e43b:7316 prefixlen 64 scopeid 0x20<link>
    ether a8:93:4a:5f:50:dd txqueuelen 1000 (Ethernet)
    RX packets 376 bytes 365931 (365.9 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 350 bytes 40198 (40.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

caccolillo@caccolillo-OMEN-25L-Desktop-GT12-1xxx:~$ ifconfig enp5s0 192.168.1.10
SIOCSIFADDR: Operation not permitted
SIOCSIFFLAGS: Operation not permitted
caccolillo@caccolillo-OMEN-25L-Desktop-GT12-1xxx:~$ sudo ifconfig enp5s0 192.168.1.10
[sudo] password for caccolillo:
caccolillo@caccolillo-OMEN-25L-Desktop-GT12-1xxx:~$ ifconfig
enp5s0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 192.168.1.10 netmask 255.255.255.0 broadcast 192.168.1.255
    ether 6c:02:e0:45:16:fb txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
```

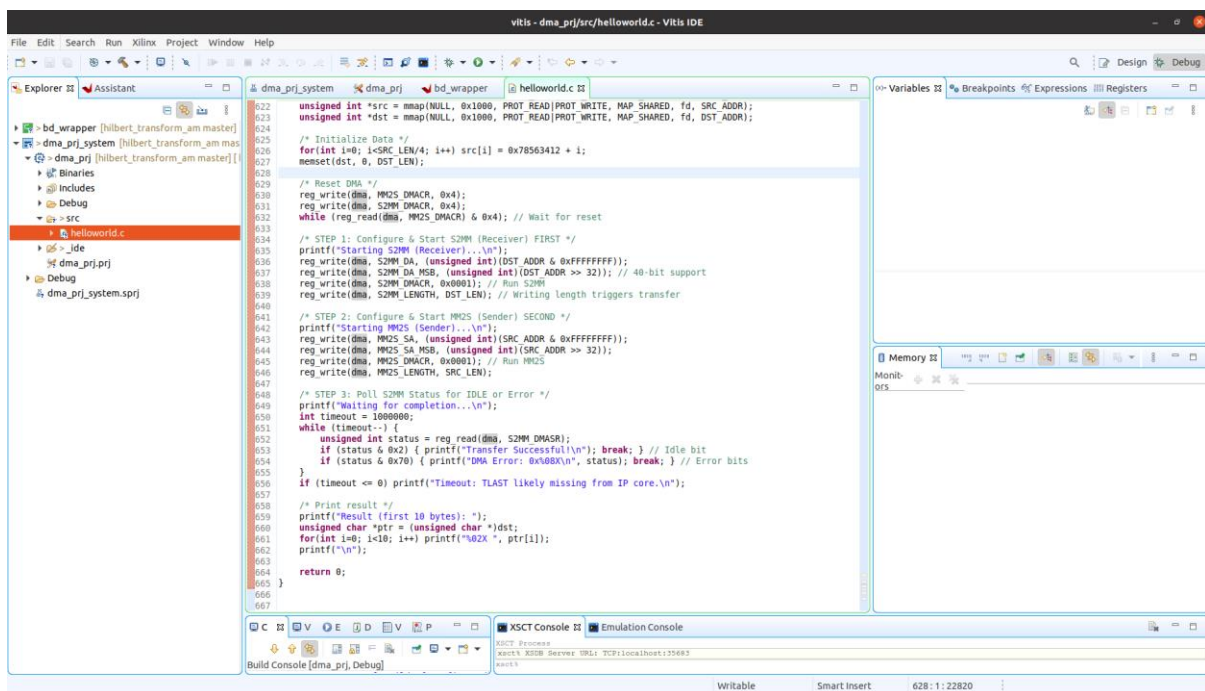
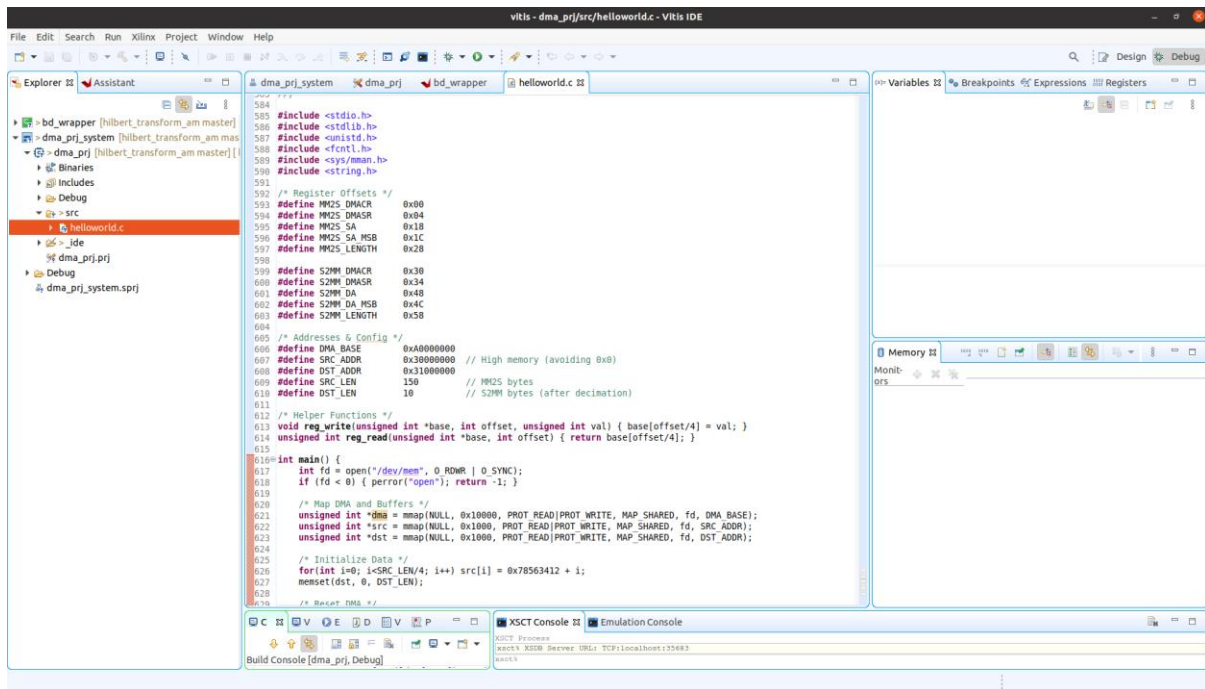
THE LINUX SW

The Linux SW can be developed in Vitis as per the following tutorials:

<https://xilinx.github.io/Embedded-Design-Tutorials/docs/2021.2/build/html/docs/Introduction/Zynq7000-EDT/4-linux-for-zynq.html>

<https://www.youtube.com/watch?v=a9ozwIT98rc>

The following test program has been prepared:



The purpose of this application is to validate the correct operation of the AXI DMA engine and the associated streaming data path, including any intermediate processing IP cores. The example demonstrates a complete end-to-end data transfer without relying on kernel-space drivers.

The application accesses the AXI DMA control interface via /dev/mem, allowing direct userspace interaction with physical memory addresses. Data is transferred from a source buffer in DDR memory to a destination buffer through the AXI Stream interface, enabling real-time processing via the Hilbert based demodulator IP-core.

The S2MM (receiver) channel is configured and started before the MM2S (transmitter) channel to ensure that the streaming sink is ready to accept incoming data. Writing to the DMA length registers triggers the respective transfers.

The source buffer is initialized with a known test pattern to allow easy verification of data integrity after the transfer. The destination buffer is cleared prior to the DMA operation to ensure that only newly received data is evaluated.

Transfer completion is detected by polling the S2MM status register and monitoring the idle flag. Error conditions such as internal DMA faults or stream errors are detected by checking the DMA status error bits.

A timeout mechanism is implemented to prevent indefinite blocking in the event of a transfer failure. A timeout condition typically indicates a missing or improperly generated TLAST signal from the streaming IP core.

Upon successful completion of the DMA transfer, the first bytes of the destination buffer are printed for verification. This confirms correct data movement through the AXI DMA and the streaming data path.

This userspace DMA example provides a simple and effective method for validating AXI DMA functionality and stream-based IP integration during early system bring-up and debugging phases.

By launching the program from the terminal on the Ultra96 board, we get:

```
GTKTerm - /dev/ttyUSB1 115200-8-N-1
File Edit Log Configuration Controlsignals View Help
18.753380] sda: sda1
18.761783] sd 0:0:0:0: [sda] Attached SCSI removable disk
18.873624] blk update request: I/O error, dev mmcblk1, sector 1956368 op 0x0:(READ) flags 0x00700 phys_seg 19 prio class 0
19.727272] FAT-fs (sda1): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.
19.824673] blk update request: I/O error, dev mmcblk1, sector 1956368 op 0x0:(READ) flags 0x00700 phys_seg 31 prio class 0
20.768469] blk update request: I/O error, dev mmcblk1, sector 1956368 op 0x0:(READ) flags 0x00700 phys_seg 49 prio class 0
21.348467] blk update request: I/O error, dev mmcblk1, sector 1956368 op 0x0:(READ) flags 0x00700 phys_seg 52 prio class 0
21.848471] blk update request: I/O error, dev mmcblk1, sector 1956368 op 0x0:(READ) flags 0x00700 phys_seg 57 prio class 0
22.034290] ax88179_178a 1-1-1:1.0 eth0: ax88179 - Link status is: 1
22.039532] IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
22.223499] EXT4-fs (mmcblk1p2): mounted filesystem with ordered data mode. Opts: (null). Quota mode: none.
94.810144] audit: type=1806 audit(1637342450.112:2): pid=780 uid=0 old-auid=4294967295 auid=1000 tty=(none) old-ses=4294967295 ses=1 res=1
94.810151] audit: type=1390 audit(1637342450.112:2): arch=c0000007 syscall=64 success=yes exit=0 a0=0 a1=ffffd7331780 a2=4 a3=ffff9344a6b0 items=0 ppid=1 pid=780 auid=1000 uid=0 gl
d=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=1 comm="(systemd)" exe="/lib/systemd/systemd" key=(null)
94.820482] audit: type=1327 audit(1637342450.112:2): proctitle="(systemd)"
1039.596519] usb 1-1.2: USB disconnect, device number 5
1041.518044] usb 1-1.2: new high-speed USB device number 6 using xhci-hcd
1041.635120] usb 1-1.2: New USB device found, idVendor=03f0, idProduct=a240, bcdDevice= 1.00
1041.635139] usb 1-1.2: New USB device strings: Mfr=1, Product=2, SerialNumber=3
1041.635146] usb 1-1.2: Product: v212w
1041.635152] usb 1-1.2: Manufacturer: HP
1041.635157] usb 1-1.2: SerialNumber: 878A986F393E3C67
1041.640640] usb-storage 1-1.2:1.0: USB Mass Storage device detected
1041.641147] scsi host0: usb-storage 1-1.2:1.0
1045.425492] scsi 0:0:0:0: Direct-Access hp v212w PMAP PQ: 0 ANSI: 6
1045.433151] sd 0:0:0:0: [sda] 30382208 512-byte logical blocks: (15.5 GB/14.4 GiB)
1045.437982] sd 0:0:0:0: [sda] Write Protect is off
1045.437939] sd 0:0:0:0: [sda] Mode Sense: 23 00 00 00
1045.439749] sd 0:0:0:0: [sda] No Caching mode page found
1045.445320] sd 0:0:0:0: [sda] Assuming drive cache: write through
1045.460172] sda: sda1
1045.476525] sd 0:0:0:0: [sda] Attached SCSI removable disk
1046.462047] FAT-fs (sda1): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.
ultra96:/home/petalinux# mount /dev/sda1 /home/petalinux/mnt/
ultra96:/home/petalinux# cd /home/petalinux/mnt/
ultra96:/home/petalinux/mnt# ls
BOOTEX.LOG images model_dir
System Volume Information kagglecatsanddogs_5340 resultguide.json
cat_detector mnist run_model.py
dma model compiled.xmodel
ultra96:/home/petalinux/mnt# cd dma
ultra96:/home/petalinux/mnt/dma# ls
dma.prj.elf
ultra96:/home/petalinux/mnt/dma# ./dma.prj.elf
Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful
Result (first 10 bytes): 00 00 43 02 35 14 A7 24 18 2C
ultra96:/home/petalinux/mnt/dma#
```

Adding a loop to perform several transfers back to back works as well:

```
GTKTerm - /dev/ttyUSB1 115200-8-N-1
File Edit Log Configuration Controlsignals View Help
Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 10 bytes): DA 07 BC 07 9E 07 80 07 62 07
Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 10 bytes): 44 07 26 07 08 07 EA 06 CC 06
Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 10 bytes): AE 06 90 06 72 06 54 06 36 06
Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 10 bytes): 18 06 FC 05 ED 05 0E 05 CF 05
Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 10 bytes): C0 05 B1 05 A2 05 93 05 84 05
Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 10 bytes): 75 05 66 05 57 05 48 05 39 05
Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 10 bytes): 2A 05 1B 05 0C 05 FD 04 EE 04
Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 10 bytes): DF 04 D0 04 C1 04 B2 04 A3 04
Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 10 bytes): 94 04 85 04 76 04 67 04 58 04
Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 10 bytes): 49 04 3A 04 2B 04 1C 04 0D 04
/dev/ttyUSB1 115200-8-N-1
DTR RTS CTS CD DSR RI
```

This means the IP-core alongside the DMA infrastructure, has been correctly integrated into a live Linux system.

Details about how data is read from memory as 128 bits and is assembled into 16 bits stream input samples is shown below:



The 128 bit data is split in consecutive chunks of 16 bits starting from LSB to MSB.

THE WAVEFORM GENERATOR IN SIMULINK OVER ETHERNET

The AM Hilbert demodulator expects input data as:

$$\text{ap_fixed}\langle 16,4 \rangle = 1 \text{ sign bit} + 3 \text{ integer bits} + 12 \text{ fractional bits}$$

So when we feed it, input data has to stay within the real range :

$$-8.0 \leq \text{value} \leq +7.999755859375$$

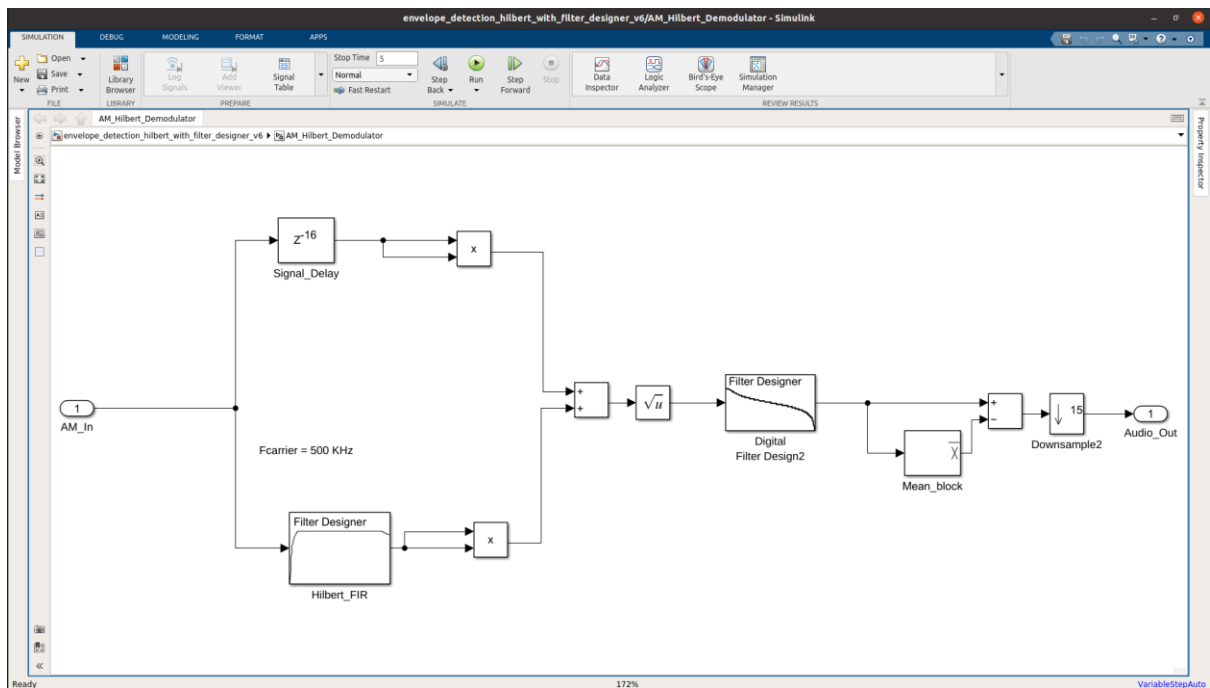
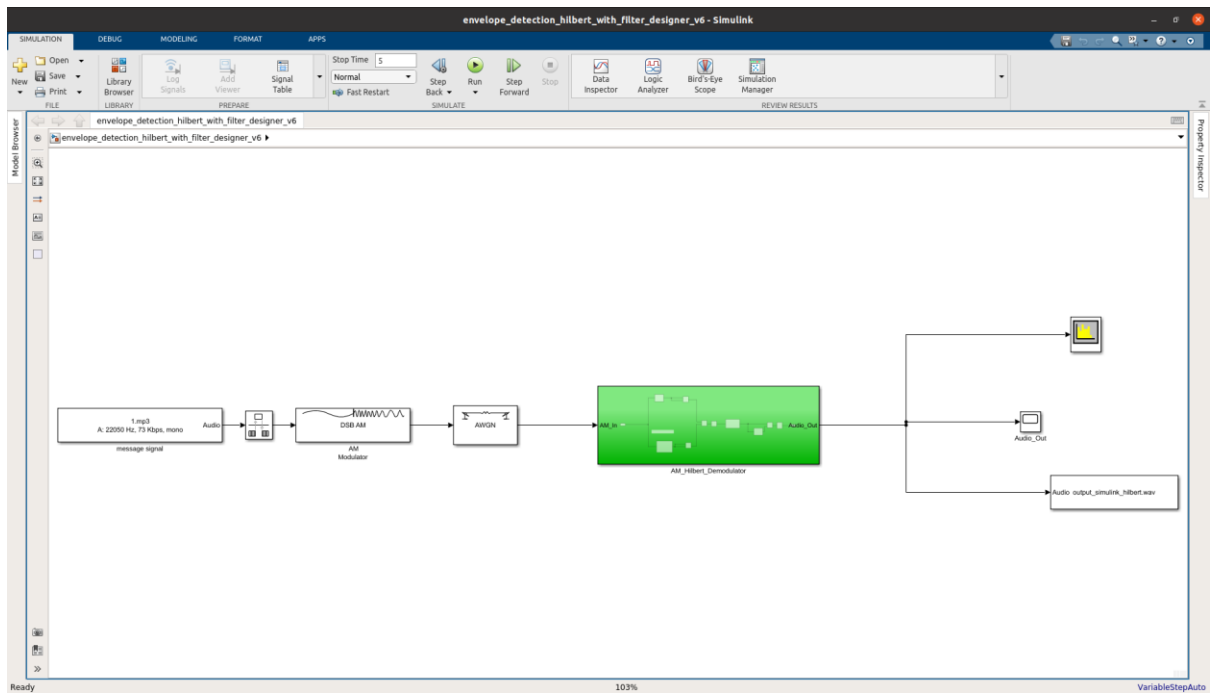
The LSB is:

$$\text{LSB} = 2^{-12} = 1 / 4096 \approx 0.000244140625$$

The signed integer limits are:

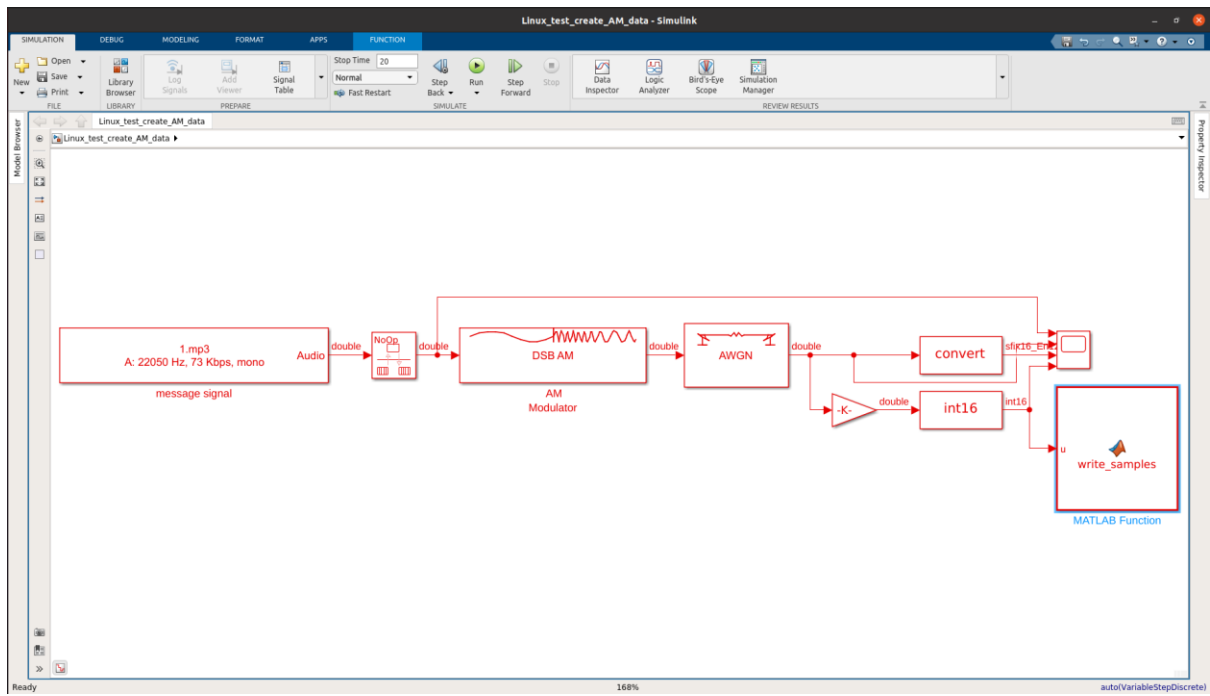
$$-8.0 \times 4096 = -32768 \quad +7.99975586 \times 4096 \approx +32767$$

The starting Simulink model has been arranged in a hierarchical manner:



To further test the behaviour of the IP-core in the Linux environment, we can think of using the very same input samples generated in the model above to exercise the IP-core.

The model has been modified as follows in order to create an input txt file with the needed samples generated in the same manner shown above:



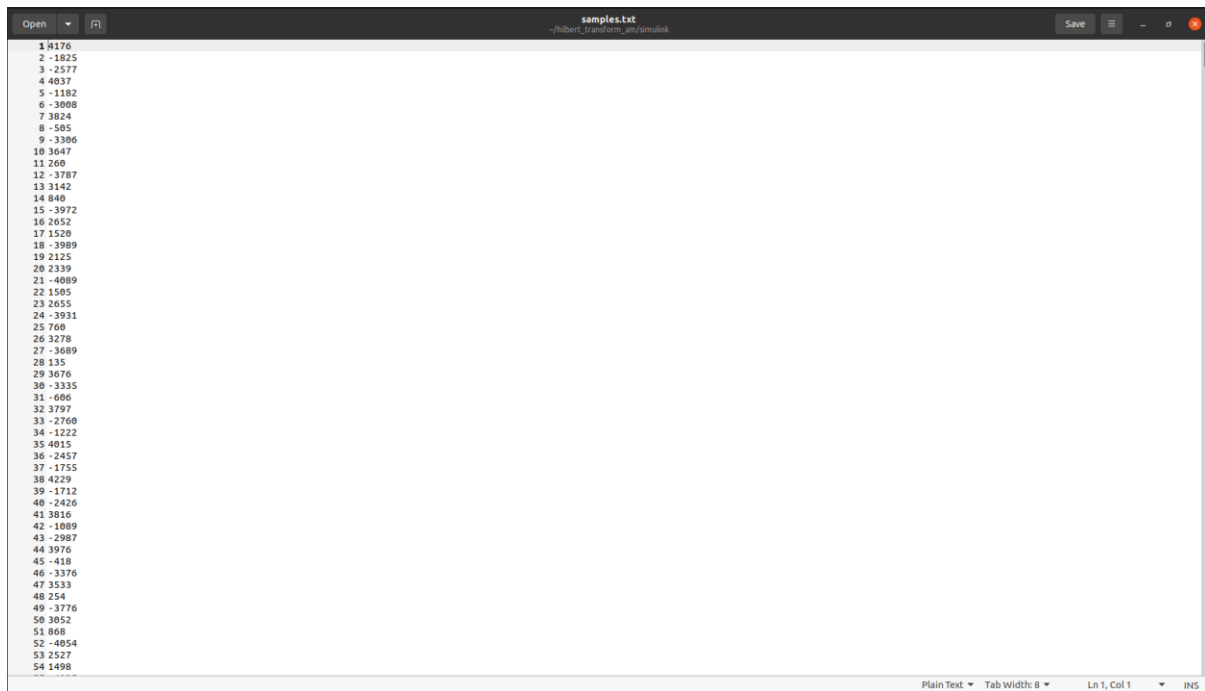
Linux_test_create_AM_data/MATLAB Function - Simulink

```

1 function write_samples(u)
2 %codegen
3 persistent fid
4 if isempty(fid)
5     fid = fopen('samples.txt','w');
6 end
7 sprintf(fid, '%d\n', int16(u));
8

```

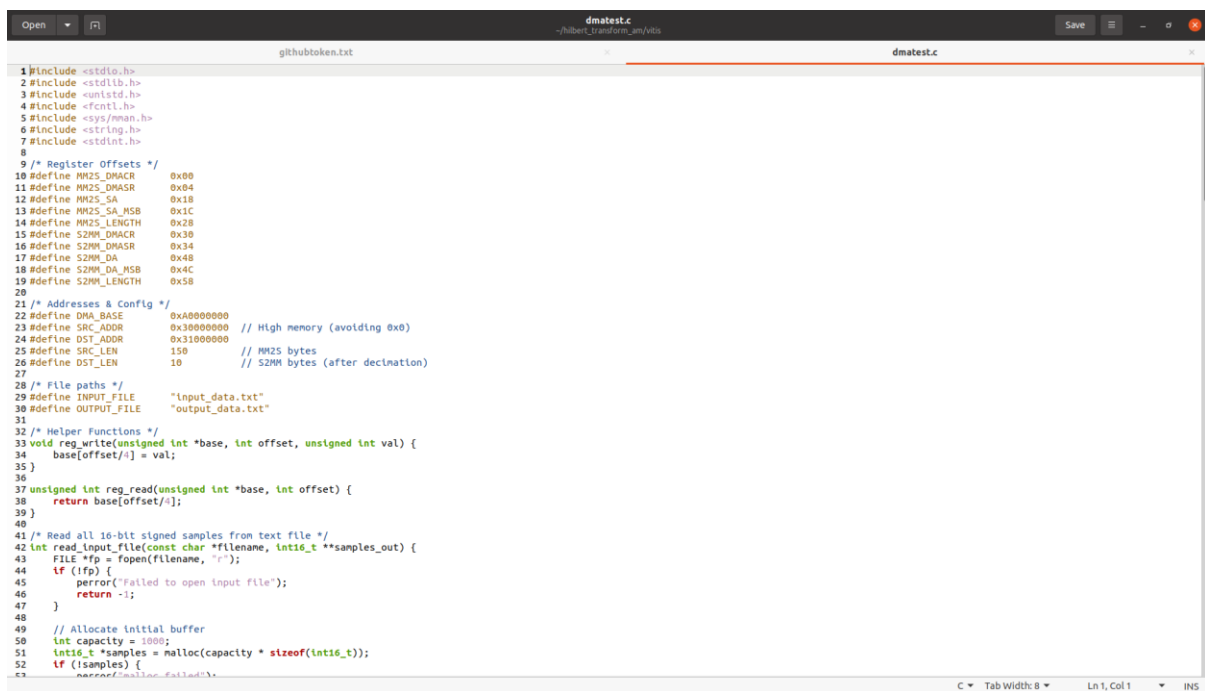
At the end of the simulation, the sought after file gets created:



The screenshot shows a text editor window titled "sample.txt" with a file path of "~libert_transform_an/simulink". The editor contains a list of 54 memory addresses, each preceded by a line number from 1 to 54. The addresses are: 1 1176, 2 -1825, 3 -2577, 4 4037, 5 -1182, 6 -3008, 7 3824, 8 -505, 9 -3396, 10 3647, 11 260, 12 -3787, 13 3142, 14 840, 15 -3972, 16 2652, 17 1528, 18 -3989, 19 2125, 20 2339, 21 -4089, 22 1505, 23 2655, 24 -3931, 25 760, 26 3278, 27 -3689, 28 135, 29 3676, 30 -3335, 31 -606, 32 3797, 33 -2760, 34 -1222, 35 4015, 36 -2457, 37 -1755, 38 4229, 39 -1712, 40 -2426, 41 3816, 42 -1089, 43 -2987, 44 3976, 45 -418, 46 -3376, 47 3533, 48 254, 49 -3776, 50 3052, 51 808, 52 -4054, 53 2527, 54 1498. The status bar at the bottom indicates "Plain Text", "Tab Width: 8", "Ln 1, Col 1", and "INS".

It shall be used to populate the source memory buffer seen before in the Linux program discussed previously (to be modified for this purpose).

The Linux C code, has been changed into the code below:



The screenshot shows a C code editor window titled "dmatest.c" with a file path of "~libert_transform_an/vis". The editor contains C code for a memory buffer test. The code includes headers for stdio.h, stdlib.h,unistd.h,fcntl.h,sys/mman.h, and string.h. It defines register offsets for MM25 and S2MM registers, addresses for DMA, SRC, and DST, and file paths for input and output data. The code also defines helper functions for writing to and reading from a register, and a function to read all 16-bit signed samples from a text file. The code is commented with /* and */. The status bar at the bottom indicates "C", "Tab Width: 8", "Ln 1, Col 1", and "INS".

```
Open  dmatetest.c  Save  -  a  x
githubtoken.txt  dmatetest.c

53     perror("malloc failed");
54     fclose(fp);
55     return -1;
56 }
57
58 int count = 0;
59 int16_t sample;
60
61 // Read all samples from file
62 while (fscanf(fp, "%hd", &sample) == 1) {
63     if (count >= capacity) {
64         // Expand buffer if needed
65         capacity *= 2;
66         int16_t *new_samples = realloc(samples, capacity * sizeof(int16_t));
67         if (new_samples) {
68             perror("realloc failed");
69             free(samples);
70             fclose(fp);
71             return -1;
72         }
73         samples = new_samples;
74     }
75     samples[count++] = sample;
76 }
77
78 fclose(fp);
79 *samples_out = samples;
80 printf("Read %d samples from %s\n", count, filename);
81 return count;
82 }
83
84 /* Copy samples to DMA buffer, packing two 16-bit samples per 32-bit word */
85 void prepare_dma_buffer(unsigned int *buffer, int16_t *samples, int num_samples,
86                         int start_idx, int buffer_size_bytes) {
87     int num_words = buffer_size_bytes / 4; // Number of 32-bit words
88
89     // Clear entire buffer first
90     memset(buffer, 0, buffer_size_bytes);
91
92     // Pack two 16-bit samples into each 32-bit word
93     for (int i = 0; i < num_words; i++) {
94         int sample_idx0 = start_idx + (i * 2); // First sample (lower 16 bits)
95         int sample_idx1 = start_idx + (i * 2) + 1; // Second sample (upper 16 bits)
96
97         uint16_t sample0 = (sample_idx0 < num_samples) ? (uint16_t)samples[sample_idx0] : 0;
98         uint16_t sample1 = (sample_idx1 < num_samples) ? (uint16_t)samples[sample_idx1] : 0;
99
100        // Pack: [sample1 (upper 16 bits) | sample0 (lower 16 bits)]
101        buffer[i] = ((uint32_t)samples[sample_idx1] << 16) | (uint32_t)samples[sample_idx0];
102    }
103 }
104
105 /* Unpack 16-bit samples from output buffer and write to text file */
106 void write_output_file(const char *filename, unsigned char *data, int length, int append) {
107     FILE *fp = fopen(filename, append ? "a" : "w");
108     if (!fp) {
109         perror("Failed to open output file");
110         return;
111     }
112
113     int16_t *samples = (int16_t *)data;
114     int num_samples = length / 2; // Each sample is 2 bytes
115
116     for (int i = 0; i < num_samples; i++) {
117         fprintf(fp, "%d\n", samples[i]);
118     }
119
120     fclose(fp);
121 }
122
123 int main() {
124     int fd = open("/dev/mem", O_RDWR | O_SYNC);
125     if (fd < 0) {
126         perror("open");
127         return -1;
128     }
129
130     /* Map DMA and Buffers */
131     unsigned int *dma = mmap(NULL, 0x10000, PROT_READ|PROT_WRITE, MAP_SHARED, fd, DMA_BASE);
132     unsigned int *src = mmap(NULL, 0x1000, PROT_READ|PROT_WRITE, MAP_SHARED, fd, SRC_ADDR);
133     unsigned int *dst = mmap(NULL, 0x1000, PROT_READ|PROT_WRITE, MAP_SHARED, fd, DST_ADDR);
134
135     if (dma == MAP_FAILED || src == MAP_FAILED || dst == MAP_FAILED) {
136         perror("mmap");
137         close(fd);
138         return -1;
139     }
140
141     /* Read all input samples from file */
142     int16_t *input_samples = NULL;
143     int total_samples = read_input_file(INPUT_FILE, &input_samples);
144     if (total_samples < 0) {
145         munmap(dma, 0x10000);
146         munmap(src, 0x1000);
147         munmap(dst, 0x1000);
148         close(fd);
149         return -1;
150     }
151
152     /* Calculate number of iterations needed */
153     int samples_per_iteration = SRC_LEN / 2; // 150 bytes = 75 samples
154     int num_iterations = (total_samples + samples_per_iteration - 1) / samples_per_iteration;
155 }
```

```
Open  ghubtoken.txt  dmatet.c  Save  -  d  x
~/hubert_transform_an/vis  dmatet.c

157 printf("Samples per iteration: %d\n", samples_per_iteration);
158 printf("Number of iterations: %d\n", num_iterations);
159
160 /* Clear output file */
161 FILE *fp = fopen(OUTPUT_FILE, "w");
162 if (fp) fclose(fp);
163
164 for(int j = 0; j < num_iterations; j++){
165     /* Prepare DMA source buffer with samples (pad with zeros if needed) */
166     int start_idx = j * samples_per_iteration;
167     prepare_dma_buffer(src, input_samples, total_samples, start_idx, SRC_LEN);
168
169     /* Initialize destination buffer */
170     memset(dst, 0, DST_LEN);
171
172     /* Reset DMA */
173     reg_write(dma, MM2S_DMAR, 0x4);
174     reg_write(dma, S2MM_DMAR, 0x0);
175     while (reg_read(dma, MM2S_DMAR) & 0x4); // Wait for reset
176
177     /* STEP 1: Configure & Start S2MM (Receiver) FIRST */
178     printf("Iteration %d/%d: Starting S2MM (Receiver)...\n", j+1, num_iterations);
179     reg_write(dma, S2MM_DA, (unsigned int)(SRC_ADDR & 0xFFFFFFFF));
180     reg_write(dma, S2MM_DA_MSB, (unsigned int)(DST_ADDR >> 32)); // 40-bit support
181     reg_write(dma, S2MM_DMAR, 0x0001); // Run S2MM
182     reg_write(dma, S2MM_LENGTH, DST_LEN); // Writing length triggers transfer
183
184     /* STEP 2: Configure & Start MM2S (Sender) SECOND */
185     printf("Starting MM2S (Sender)...\n");
186     reg_write(dma, MM2S_SA, (unsigned int)(SRC_ADDR & 0xFFFFFFFF));
187     reg_write(dma, MM2S_SA_MSB, (unsigned int)(SRC_ADDR >> 32));
188     reg_write(dma, MM2S_DMAR, 0x0001); // Run MM2S
189     reg_write(dma, MM2S_LENGTH, SRC_LEN);
190
191     /* STEP 3: Poll S2MM Status for IDLE or Error */
192     printf("Waiting for completion...\n");
193     int timeout = 1000000;
194     while (timeout-- > 0) {
195         unsigned int status = reg_read(dma, S2MM_DMAR);
196         if (status & 0x2) {
197             printf("Transfer Successful!\n");
198             break;
199         } // Idle bit
200         if (status & 0x70) {
201             printf("DMA Error: 0x%08X\n", status);
202             break;
203         } // Error bits
204     }
205
206     if (timeout <= 0)
207         printf("Timeout: TLAST likely missing from IP core.\n");
208
209     /* Print result */
210     printf("Result (first %d samples): ", DST_LEN/2);
211     int16_t *samples = (int16_t *)dst;
212     for(int i = 0; i < DST_LEN/2; i++) printf("%d ", samples[i]);
213     printf("\n\n");
214
215     /* Write output to file (append mode) */
216     write_output_file(OUTPUT_FILE, (unsigned char *)dst, DST_LEN, (j > 0));
217 }
218
219 printf("Output saved to %s\n", OUTPUT_FILE);
220
221 /* Free allocated memory */
222 free(input_samples);
223
224 /* Unmap memory buffers and close /dev/mem file */
225 munmap(dma, 0x10000);
226 munmap(src, 0x10000);
227 munmap(dst, 0x10000);
228 close(fd);
229
230 return 0;
231 }
```

This C program implements a DMA (Direct Memory Access) transfer system for processing audio or signal data through a hardware accelerator on an embedded Linux system. The code reads 16-bit signed samples from an input text file, processes them through a hardware IP core via DMA transfers, and writes the decimated results to an output file.

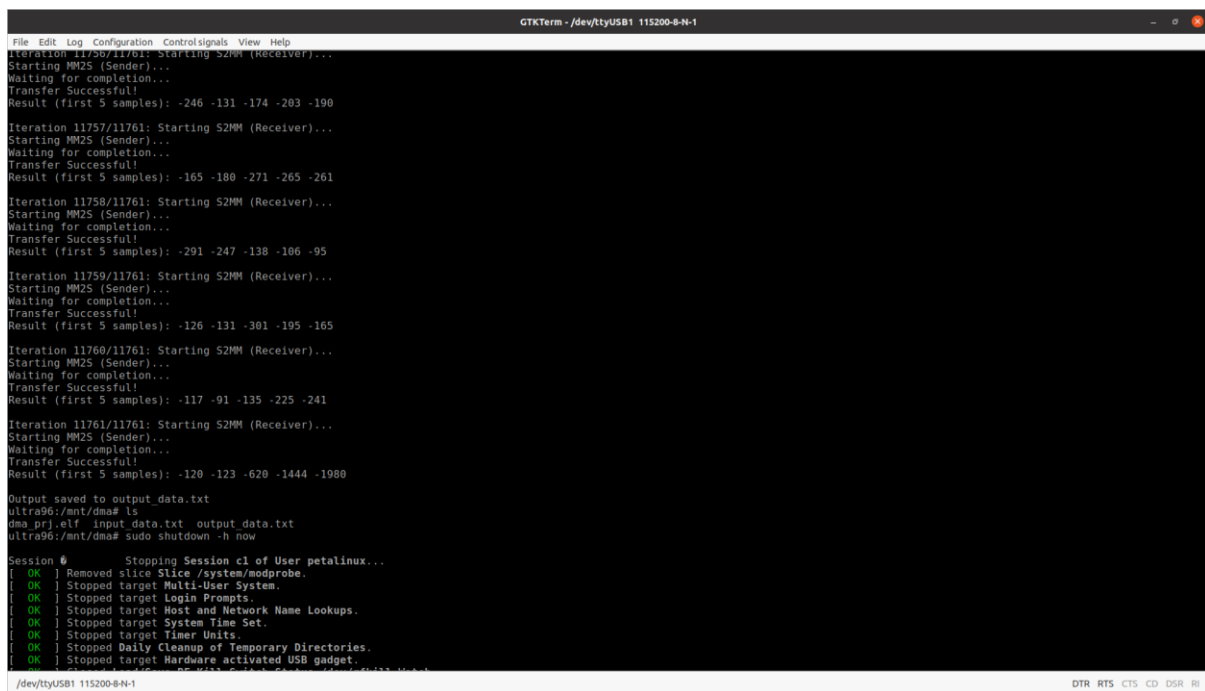
The program operates by memory-mapping physical addresses to access DMA registers and data buffers directly through /dev/mem. It reads all input samples from a text file into memory, then processes them in batches of 75 samples (150 bytes) per iteration. For each batch, the code packs two 16-bit samples into each 32-bit word and writes them to the source buffer at address 0x30000000.

The DMA transfer follows a specific sequence where the receive channel (S2MM) is configured and started first, followed by the transmit channel (MM2S). This ordering ensures the receiver is ready before data arrives. The MM2S channel reads 150 bytes from the source buffer and streams them through the hardware IP core, which appears to perform a decimation operation that reduces the data by a factor of 15, producing 10

bytes of output that the S2MM channel writes to the destination buffer at address 0x31000000.

After each transfer completes, the program unpacks the resulting 16-bit samples and appends them to the output text file. The process repeats until all input samples have been processed, with the final iteration padding any remaining samples with zeros if needed. The code includes timeout handling and error checking to detect DMA failures, though it notes that timeouts may occur if the IP core doesn't properly assert the TLAST signal to indicate the end of a stream.

By running this code on the Ultra96 V2 board:



```
GTKTerm - /dev/ttyUSB1 115200-8-N-1
File Edit Log Configuration Controls/Signals View Help
Iteration 11756/11761: Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 5 samples): -246 -131 -174 -203 -190

Iteration 11757/11761: Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 5 samples): -165 -180 -271 -265 -261

Iteration 11758/11761: Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 5 samples): -291 -247 -138 -106 -95

Iteration 11759/11761: Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 5 samples): -126 -131 -301 -195 -165

Iteration 11760/11761: Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 5 samples): -117 -91 -135 -225 -241

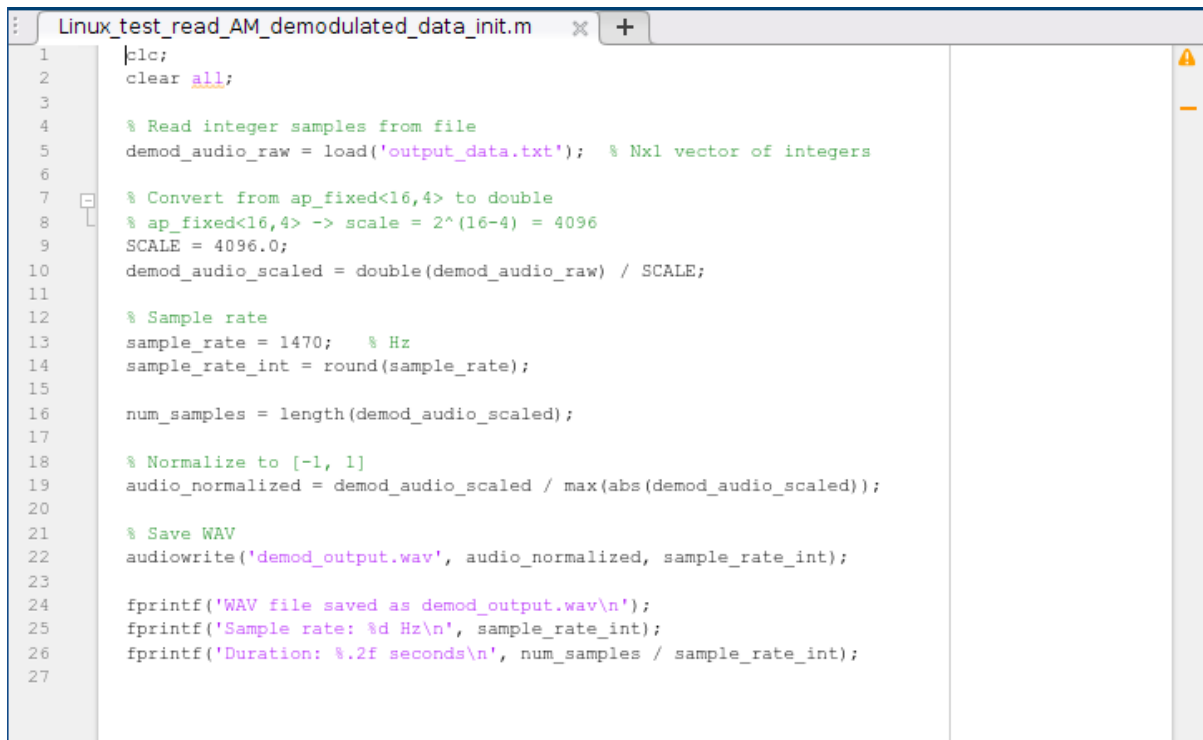
Iteration 11761/11761: Starting S2MM (Receiver)...
Starting MM2S (Sender)...
Waiting for completion...
Transfer Successful!
Result (first 5 samples): -120 -123 -620 -1444 -1980

Output saved to output_data.txt
ultra96:/mnt/dma# ls
dma.prj.elf input_data.txt output_data.txt
ultra96:/mnt/dma# sudo shutdown -h now

Session #          Stopping Session c1 of User petalinux...
[ OK ] Removed slice /system/modprobe.
[ OK ] Stopped target Multi-User System.
[ OK ] Stopped target Login Prompts.
[ OK ] Stopped target Host and Network Name Lookups.
[ OK ] Stopped target System Time Set.
[ OK ] Stopped target Timer Units.
[ OK ] Stopped Daily Cleanup of Temporary Directories.
[ OK ] Stopped target Hardware activated USB gadget.

/dev/ttyUSB1 115200-8-N-1          DTR RTS CTS CD DSR RI
```

When fed by the input file created via the Simulink model, it creates the output text file with the demodulated signal, that can be post processed with the Matlab script below, and saved as a WAV file:



```
1  clc;
2  clear all;
3
4  % Read integer samples from file
5  demod_audio_raw = load('output_data.txt'); % Nx1 vector of integers
6
7  % Convert from ap_fixed<16,4> to double
8  % ap_fixed<16,4> -> scale = 2^(16-4) = 4096
9  SCALE = 4096.0;
10 demod_audio_scaled = double(demod_audio_raw) / SCALE;
11
12 % Sample rate
13 sample_rate = 1470; % Hz
14 sample_rate_int = round(sample_rate);
15
16 num_samples = length(demod_audio_scaled);
17
18 % Normalize to [-1, 1]
19 audio_normalized = demod_audio_scaled / max(abs(demod_audio_scaled));
20
21 % Save WAV
22 audiowrite('demod_output.wav', audio_normalized, sample_rate_int);
23
24 fprintf('WAV file saved as demod_output.wav\n');
25 fprintf('Sample rate: %d Hz\n', sample_rate_int);
26 fprintf('Duration: %.2f seconds\n', num_samples / sample_rate_int);
27
```

Playing the WAV file, we can hear back what has been transmitted.

The audio quality is not excellent, but the speech can be heard.

The design hasn't been improved thoroughly as it is was meant only as a proof of concept.

CONCLUSIONS

This work demonstrates a complete design flow for implementing a Hilbert transform based AM demodulator, successfully bridging the gap between algorithm development and hardware deployment. The systematic progression from Simulink modeling through Vitis HLS implementation to FPGA integration on a Zynq UltraScale+ platform validates the robustness of the design methodology. Each stage of verification—from individual HLS component testbenches to AXI Stream VIP simulations and ultimately system-level validation with DMA transfers—confirmed correct operation and uncovered integration challenges early in the development cycle. The final deployment on the Ultra96 V2 board running PetaLinux, with successful processing of Simulink-generated test signals through userspace DMA control, proves the design's viability for real-world embedded applications. The modular architecture, combining delay compensation, Hilbert transformation, envelope detection with Newton-Raphson approximation, IIR filtering, and decimation, achieves effective AM demodulation while maintaining reasonable resource utilization. This implementation serves as a practical reference for signal processing IP core development, demonstrating that complex DSP algorithms can be

efficiently translated from simulation environments to production embedded systems through careful verification at each abstraction level.